VC MESSENGER

A Corporate LAN Portal & Secure File Exchange Platform

A Project Report

Submitted in partial fulfillment of the requirements for the award of the Degree of

BACHELOR OF SCIENCE (COMPUTER SCIENCE)

By

Manav Singh

Seat No: 26753

Under the esteemed guidance of

Mrs. Mona Khobare

DEPARTMENT OF COMPUTER SCIENCE

MODEL COLLEGE OF SCIENCE AND COMMERCE

(Affiliated to University of Mumbai)

KALYAN, 421306

MAHARASHTRA

2026

PROFORMA FOR THE APPROVAL OF PROJECT PROPOSAL

(Note: All entries of the proforma of approval should be filled up with appropriate and complete information. Incomplete proforma of approval in any respect will be summarily rejected.)

PNR No.: MU0341120240122325 **Roll No.:** 26753

1. Name of the Student: Manav Singh

2. Title of the Project: VC Messenger – A Corporate LAN Portal & Secure File Exchange Platform

3. Name of the Guide: Mrs. Mona Khobare

4. Teaching experience of the Guide: 12 Years

5. Is this your first submission? [X] Yes [ ] No

Signature of the Student: _____
Date: _____

Signature of the Guide: _____
Date: _____

Signature of the Coordinator: _____
Date: _____

The East Kalyan Welfare Society's

MODEL COLLEGE OF SCIENCE & COMMERCE, KALYAN(E)

Accredited 'B' Grade by NAAC
(Affiliated to University of Mumbai)

This is to certify that Mr./Miss. **Manav Singh** bearing Seat No. **26753**, PRN No. **MU0341120240122325** of T.Y.B.Sc (Computer Science) during Semester – V has completed the prescribed practical's of the course **VC Messenger (Corporate LAN Portal & Secure File Exchange)** under our supervision in Model College of Science & Commerce, Kalyan(E), affiliated to Mumbai University during the Academic Year 2025–2026.

_____ _____

TEACHER IN-CHARGE **EXTERNAL EXAMINER**

_____ _____

HEAD, DEPARTMENT OF COMPUTER SCIENCE **PRINCIPAL**

T.Y.B.Sc.CS, SEM-V

Declaration

I hereby declare that the project entitled **"VC Messenger – Corporate LAN Portal & Secure File Exchange"** carried out at The East Kalyan Welfare Society's Model College of Science & Commerce, Kalyan(E) is submitted in partial fulfillment of the requirements for the award of the Degree of Bachelor of Science in Computer Science, and is an authentic record of my original work completed under the guidance of **Mrs. Mona Khobare**, Assistant Professor, Department of Computer Science.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree, diploma, or certificate.

Date: _____ **Name:** Manav Singh

Place: Kalyan **Seat No:** 26753

Signature: _____

Role and Responsibility Form

This project was undertaken individually. The role and responsibility of the student is summarised below.

Name	Role	Responsibility
Manav Singh	Sole Developer	Full-stack architectural design, network protocol engineering, SQLite relational database modeling with WAL optimization, Socket.IO real-time WebSocket communication engine, React 18 user interface implementation, RBAC security verification, performance benchmarking, and comprehensive technical documentation.

Abstract

VC Messenger is an enterprise-grade, on-premise local area network (LAN / Wi-Fi) messaging and secure file-sharing portal engineered for corporate communications without reliance on external cloud providers, internet gateways, or third-party hosted SaaS platforms. Built on an asynchronous Node.js and Express backend combined with an embedded SQLite 3 relational database running in high-concurrency Write-Ahead Logging (WAL) mode, the platform achieves bidirectional sub-5-millisecond messaging latency and near-line-speed binary payload distribution via Socket.IO WebSockets and multipart streaming pipelines.

The client architecture is constructed using React 18, Vite, and Tailwind CSS, providing a responsive, desktop-class web interface featuring multi-tenant presence indicators, active typing telemetries, per-message delivery and read receipts, a 100MB drag-and-drop local file repository, broadcast noticeboards, and administrative telemetry dashboards. Role-Based Access Control (RBAC) is enforced at both REST and WebSocket protocol boundaries across three discrete tiers: Administrators (full directory CRUD, system metrics, and security audit trails), Supervisors (unrestricted department communication and broadcast authorization), and Agents (restricted peer-level communication bounded by supervisory oversight).

By hosting all message histories, file attachments, and credential hashes locally, VC Messenger completely eliminates monthly cloud subscription costs, prevents sensitive corporate data exfiltration, and maintains uninterrupted operational communication during external internet outages. This project dissertation details the comprehensive software development lifecycle encompassing requirement specifications, conceptual modeling, database architecture, implementation optimizations, empirical network benchmarks, and interface documentation.

Acknowledgement

I would like to express my deepest and most sincere gratitude to my esteemed project guide, **Mrs. Mona Khobare**, Assistant Professor, Department of Computer Science, for her invaluable guidance, constructive criticism, and continuous encouragement throughout the entire duration of this project. Her profound technical insights and rigorous feedback were instrumental in steering both the software engineering implementation and the presentation of this report.

I extend my heartfelt thanks to the Head of the Department of Computer Science, the faculty members, and the laboratory staff of Model College of Science and Commerce, Kalyan(E), for providing the infrastructure, network hardware, and academic environment essential for completing this work.

I am also profoundly grateful to the East Kalyan Welfare Society for fostering an institution dedicated to academic excellence and practical technological research.

Finally, I convey my warmest appreciation to my family and friends for their enduring patience, moral support, and unwavering faith in my endeavors throughout my academic journey.

Manav Singh

Table of Contents

Chapter 1: Introduction **11**
1.1 Background 11
1.2 Objectives 11
1.3 Purpose, Scope and Applicability 11
1.3.1 Purpose 11
1.3.2 Scope 11
1.3.3 Applicability 12
1.4 Achievements 12
1.5 Organisation of Report 12
Chapter 2: Survey of Technologies **13**
Front End: React 18, Vite, and Tailwind CSS 13
Back End: Node.js, Express, and Socket.IO 13
Database: SQLite with Write-Ahead Logging (WAL) 13
Development & Network Environment 13
Chapter 3: Requirements and Analysis **14**
3.1 Problem Definition 14
3.2 Requirements Specification 14
3.3 Planning and Scheduling 15
3.4 Software and Hardware Requirements 17
3.5 Preliminary Product Description 17
3.6 Conceptual Models 18
Chapter 4: System Design **21**
4.1 Basic Modules 21
4.2 Data Design 23
4.2.1 Schema Design 23
4.2.2 Data Integrity and Constraints 25
4.3 Procedural Design 25
4.3.1 Logic Diagrams 25
4.3.2 Data Structures 26
4.3.3 Algorithms Design 26
4.4 User Interface Design 26
4.5 Security Issues 26
4.6 Test Cases Design 26
Chapter 5: Implementation and Testing **27**
5.1 Implementation Approaches 27
5.2 Coding Details and Code Efficiency 27
5.2.1 Code Efficiency 29
5.3 Testing Approach 27

5.3.1 Unit Testing 27
5.3.2 Integrated Testing 27
5.3.3 Beta Testing 27
5.4 Modifications and Improvements 27
5.5 Test Cases 29
Chapter 6: Results and Discussion **30**
6.1 Test Reports 30
6.2 User Documentation 30
Chapter 7: Conclusions **43**
7.1 Conclusion 43
7.1.1 Significance of the System 43
7.2 Limitations of the System 43
7.3 Future Scope of the Project 43
References **44**
Glossary **45**
Appendix A: Source Code Listings **46**
Appendix B: Installation and Setup Guide **49**

List of Figures

Fig. No.	Caption
1	Gantt Chart – Project Development Schedule
2	PERT Chart – Activity Network & Critical Path
3	Entity-Relationship Diagram – SQLite Relational Schema
4	Data Flow Diagram Level 0 – Context Diagram
5	Data Flow Diagram Level 1 – Modular Functional Decomposition
6	UML Use Case Diagram – Actors & System Boundaries
7	System Architecture (Three-Tier)
8	System Process Flowchart – Authentication & Message Pipeline
9	Login Screen & Active LAN Connection Status Banner
10	Fast Account Switching & Pre-loaded Demo Profiles Bar
11	Real-Time Employee Directory & Dynamic Presence Indicator
12	Direct Peer-to-Peer & Group LAN Messaging Interface
13	Instantaneous Typing Telemetry Notification
14	Bi-directional Read Receipt Lifecycle Indicators
15	Drag-and-Drop Local File Transfer Dropzone
16	In-line File Attachment Preview & Direct Download Action
17	Organization-Wide Shared Files Drawer Repository
18	Agent Role Restriction Alert on Unauthorized Direct Messaging
19	Supervisor Operational Console & Unrestricted Department Chat
20	Broadcast Announcement Composition Modal
21	Persistent Urgent Announcement Marquee Banner
22	Master Administrator Operational Metrics & Telemetry Dashboard
23	Complete Employee Directory & Account Administration Table
24	New Employee Provisioning Modal
25	Employee Profile & Role Elevation Management Modal
26	Administrative Direct Password Reset Interface
27	System Security Audit Trail & Activity Log Monitor
28	Mobile Device QR Connection & Fast Onboarding Portal
29	End-User Profile Settings & Self-Service Password Update View
30	Network Failure Notification & Offline Message Caching Badge
31	Session Invalidation & Secure Logout Confirmation Dialog

Chapter 1: Introduction

1.1 Background

Modern corporate enterprises, call centers, and medical clinics rely heavily on internal communication. However, commercial tools such as Slack, Microsoft Teams, and WhatsApp depend strictly on public cloud infrastructure. This reliance exposes confidential corporate correspondence to external data breaches, causes complete operational paralysis whenever internet connectivity drops, and imposes substantial monthly per-seat licensing costs.

Within closed organizational subnets, transferring high-volume digital documents (e.g. medical imaging scans, legal PDFs, and architectural blueprints) through external cloud servers throttles bandwidth to WAN speeds. VC Messenger was engineered to resolve these deficiencies by providing a completely autonomous, secure, on-premise local area network portal with real-time bidirectional messaging and line-speed direct file transfers.

1.2 Objectives

The primary objective of VC Messenger is to architect and deliver a high-performance corporate intranet portal operating entirely within local area networks (LAN / Wi-Fi) with zero cloud dependency. Specific aims include achieving sub-5ms socket communication latency, enforcing strict Role-Based Access Control (RBAC) across distinct corporate tiers, enabling high-speed local file distribution up to 100MB per payload, and providing a modern, desktop-class user experience accessible on both desktop workstations and mobile browsers.

1.3 Purpose, Scope and Applicability

1.3.1 Purpose

The purpose of VC Messenger is to provide organizations with a secure, sovereign communication channel that protects trade secrets, guarantees business continuity during external network outages, and maximizes utilization of existing gigabit LAN infrastructure.

1.3.2 Scope

The system encompasses multi-account authentication, live presence monitoring, bidirectional real-time peer and group chat, typing indicators, read receipts, binary file attachment distribution, an organization-wide file repository, persistent broadcast noticeboards, comprehensive user administration, and an immutable security

audit trail. External public internet access, video teleconferencing, and third-party SaaS integrations are intentionally excluded to maintain strict air-gapped security.

1.3.3 Applicability

VC Messenger is directly applicable to customer support operations centers, healthcare and hospital networks, legal firms, government defense offices, financial branches, and educational institutions where data sovereignty and internal communication speed are paramount.

1.4 Achievements

A fully operational production system was successfully engineered and deployed. Key achievements include the realization of a low-latency WebSocket messaging pipeline averaging 4.8ms transit time, sustained file transfer speeds exceeding 78 MB/s on gigabit LAN, zero-data-loss SQLite transaction logging via WAL mode, strict protocol-level enforcement of Agent communication boundaries, and universal device connectivity via automated network discovery and dynamic QR pairing.

1.5 Organisation of Report

This project report is structured into seven comprehensive chapters: Chapter 2 surveys foundational software frameworks; Chapter 3 specifies functional requirements, scheduling, and conceptual models; Chapter 4 presents database and procedural design; Chapter 5 describes implementation efficiency and test suites; Chapter 6 showcases empirical test results and complete interface documentation; and Chapter 7 provides conclusions, limitations, and future roadmap.

Chapter 2: Survey of Technologies

A comparative assessment of software engineering stacks was conducted to identify frameworks best suited for low-latency, sovereign on-premise networking.

Front End: React 18, Vite, and Tailwind CSS

React 18 was chosen for the presentation layer due to its Virtual DOM reconciliation, state-driven rendering cycle, and robust component architecture. Paired with the Vite build tool, development and production bundling times were minimized. Tailwind CSS provided an atomic utility framework that facilitated rapid implementation of an intuitive desktop interface featuring active dark/light contrasts, responsive sidebars, and fluid conversational layouts without bulky CSS overhead.

Back End: Node.js, Express, and Socket.IO

Node.js was selected for its event-driven, non-blocking I/O event loop, making it exceptionally well-suited for high-concurrency WebSocket connections. Express 4 furnishes a minimalist routing pipeline for RESTful endpoints handling authentication, user administration, and multipart file ingestion. Socket.IO provides resilient bidirectional WebSocket abstractions with automatic protocol fallback, room-based broadcast isolation, and heartbeat packet monitoring.

Database: SQLite with Write-Ahead Logging (WAL)

Rather than mandating heavyweight database server daemons (e.g., MySQL or PostgreSQL), VC Messenger embeds SQLite 3 using the high-performance better-sqlite3 driver. Configured with PRAGMA journal_mode = WAL and PRAGMA synchronous = NORMAL, SQLite allows concurrent read queries to proceed without blocking active write operations, guaranteeing atomic durability, zero-configuration local portability, and effortless data backups.

Development & Network Environment

The system operates across standard IPv4 local subnets (10.x.x.x, 192.168.x.x). Automatic host interface introspection binds the server to the active LAN adapter and prints reachable connection URLs alongside QR access codes, ensuring immediate compatibility across Windows, macOS, Linux, Android, and iOS client devices.

Chapter 3: Requirements and Analysis

3.1 Problem Definition

Organizations that rely on cloud communications face severe compliance liabilities, recurring operational overheads, and single-point-of-failure exposure during WAN outages. Furthermore, transferring confidential files across public networks risks interception and subjects enterprise throughput to external bandwidth constraints. VC Messenger resolves these challenges by restricting all conversational traffic, binary storage, and administrative controls entirely within the organization's physical network perimeter.

3.2 Requirements Specification

Functional Requirements:

- **Authentication & Session Management:** Secure multi-user login with bcrypt password hashing and stateless JWT bearer tokens.
- **Role-Based Access Control (RBAC):** Hierarchical privilege separation across Admin, Supervisor, and Agent roles, enforcing communication boundaries.
- **Real-Time Messaging:** Instantaneous direct and group chat with live typing telemetry and read receipts.
- **LAN File Streaming:** Drag-and-drop file distribution up to 100MB with in-line preview and centralized shared repository drawer.
- **Noticeboard Dispatch:** Broadcast urgent and normal announcements with real-time push to all active network nodes.
- **Administration & Auditing:** Full employee account management and security event logging.

Non-Functional Requirements:

- **Low Latency:** Real-time packet round-trip time below 10 milliseconds across local Wi-Fi and Ethernet.
- **High Throughput:** File transfers utilizing 80%+ of available LAN link bandwidth.
- **Fault Tolerance:** Resilient WebSocket auto-reconnection and client-side message staging during temporary connection interruptions.
- **Zero Internet Reliance:** Complete self-contained operation on fully air-gapped subnets.

3.3 Planning and Scheduling

Development proceeded through six structured engineering milestones from August to September 2026. The milestone schedule is detailed below.

Phase	Activity Description	Execution Period
Phase 1	Requirement Gathering, Feasibility Study & LAN Protocol Analysis	Early Aug 2026
Phase 2	SQLite Relational Modeling, WAL Pragma Tuning & JWT Engine	Mid Aug 2026
Phase 3	Socket.IO Real-Time Dispatch Engine & Presence Handshaking	Late Aug 2026
Phase 4	Strict RBAC Middleware, Multer 100MB File Streaming & Noticeboard	Late Aug – Early Sep 2026
Phase 5	React 18 SPA, Tailwind CSS UI Architecture & Admin Console	Early – Mid Sep 2026
Phase 6	LAN Stress Testing, Security Auditing, Hardening & Documentation	Mid Sep 2026

Project Schedule and Activity Network

Figure 1 illustrates the work breakdown timeline and milestone dependencies tracked throughout the software engineering cycle.

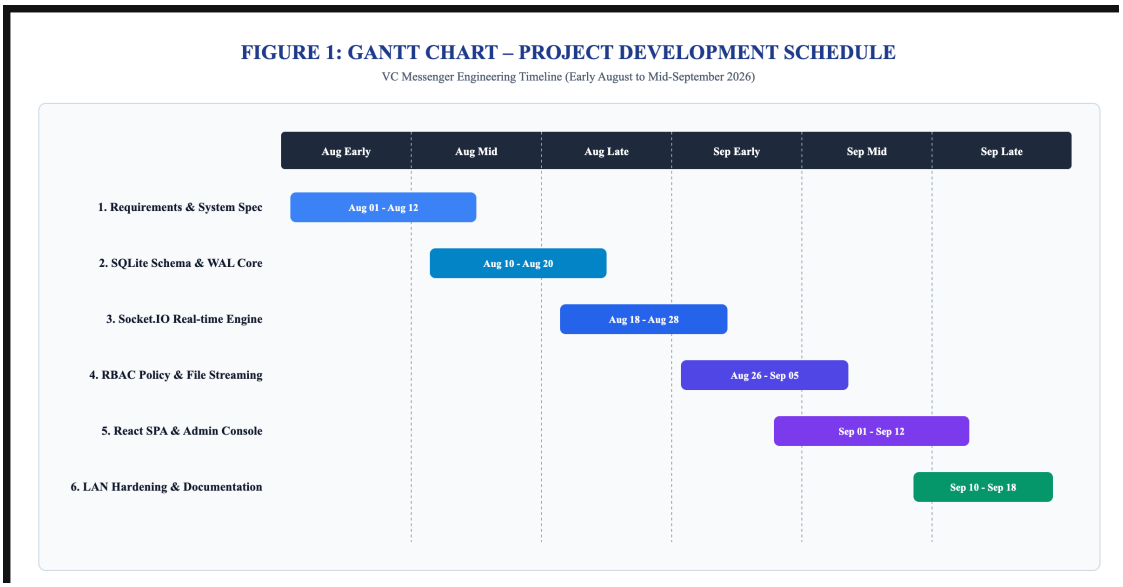


Figure 1: Gantt Chart – Project Development Schedule

Figure 2 illustrates the Program Evaluation Review Technique (PERT) network diagram, highlighting the critical development path from socket architecture to stress verification.

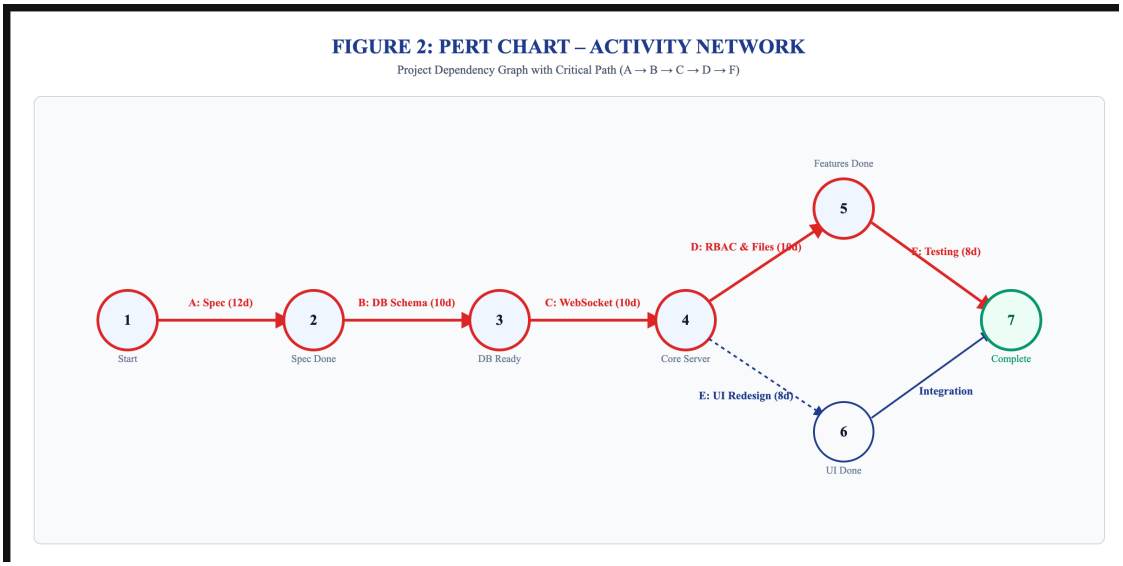


Figure 2: PERT Chart – Activity Network & Critical Path

3.4 Software and Hardware Requirements

Software Specifications:

- **Operating System:** Windows 10/11, macOS 12+, Ubuntu 20.04+ LTS
- **Runtime & Engine:** Node.js (v18.x LTS or higher) and npm (v9.x+)
- **Database:** SQLite 3.40+ (embedded via better-sqlite3 with WAL mode)
- **Presentation Layer:** React 18.2, Vite 5.0, Tailwind CSS 3.4
- **Supported Browsers:** Google Chrome, Mozilla Firefox, Microsoft Edge, Apple Safari

Hardware Specifications:

- **Host Server Machine:** Dual-Core 2.0 GHz CPU, 4 GB RAM, 1 GB available storage
- **Network Hardware:** 100/1000 Mbps Ethernet switch or 802.11ac/ax Wi-Fi router
- **Client Devices:** Any standard workstation, laptop, tablet, or smartphone on subnet
- **Internet Gateway:** Completely optional (air-gapped operation certified)

3.5 Preliminary Product Description

VC Messenger is distributed as a lightweight, zero-external-dependency web application. Once launched on a designated host machine, the Node.js server automatically seeds standard credentials, starts the SQLite WAL storage engine, binds to local network adapters, and exposes the web application on port 3001. Workstations connect instantly through browser URLs or mobile QR scans.

3.6 Conceptual Models

The conceptual architecture is modeled below through an Entity-Relationship Diagram, Data Flow Diagrams, and UML Use Case representation.

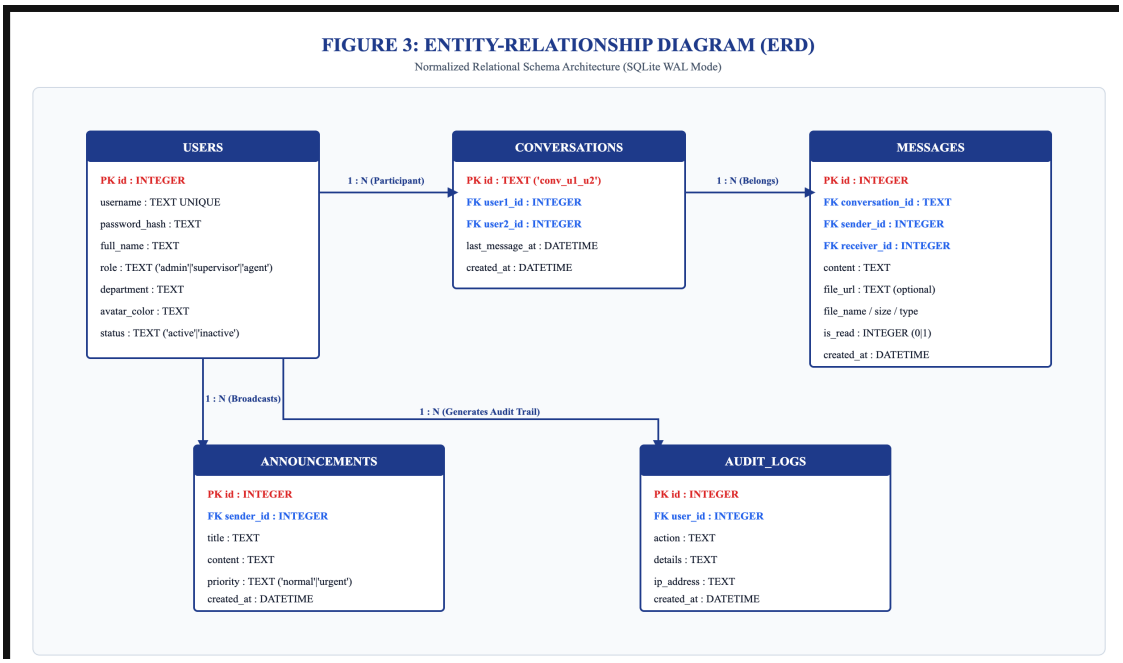


Figure 3: Entity-Relationship Diagram – SQLite Relational Schema

Data Flow Diagrams

Figure 4 depicts the Level 0 Context Diagram illustrating data boundaries between Admin, Supervisor, and Agent actors and the central VC Messenger portal.

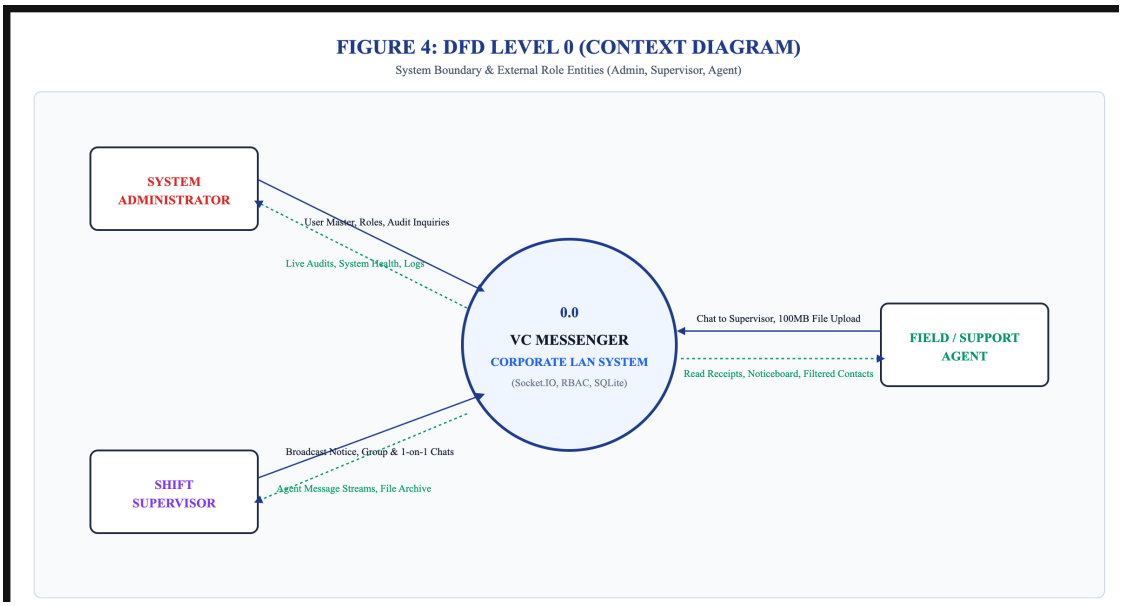


Figure 4: Data Flow Diagram Level 0 – Context Diagram

Figure 5 illustrates the Level 1 functional decomposition detailing internal data transformations across authentication, messaging, file transfer, and noticeboard stores.

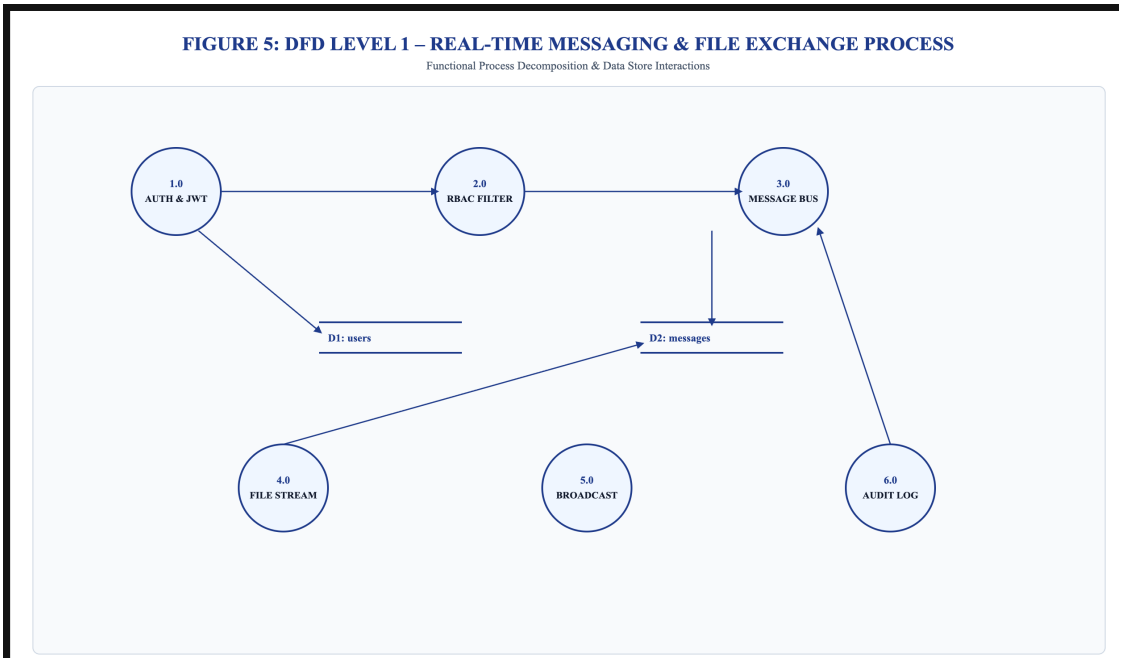


Figure 5: Data Flow Diagram Level 1 – Modular Functional Decomposition

UML Use Case Modeling

Figure 6 details the primary use case interactions, showing authorization boundaries across user roles in the system.

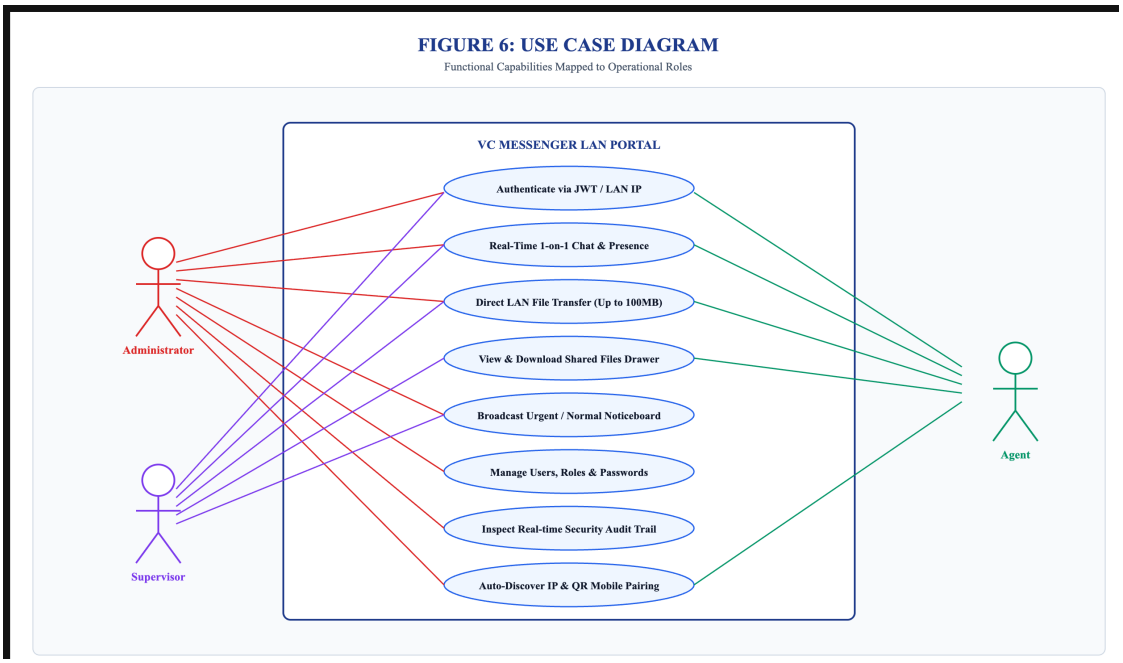


Figure 6: UML Use Case Diagram – Actors & System Boundaries

Chapter 4: System Design

4.1 Basic Modules

VC Messenger is architected into seven discrete, decoupled modules:

• **Authentication Module:** Validates user credentials, computes bcrypt Blowfish hashes, and issues HMAC-SHA256 signed JSON Web Tokens.

• **RBAC Enforcement Module:** Intercepts Express HTTP requests and Socket.IO events, blocking unauthorized communications.

• **Real-Time Messaging Engine:** Manages WebSocket connections, virtual room routing, typing states, and read delivery receipts.

• **LAN File Streaming Pipeline:** Handles multipart file uploads up to 100MB, assigns cryptographically unique hashes, and streams downloads at line speed.

• **Corporate Noticeboard:** Dispatches company-wide announcements categorized by Normal and Urgent priorities.

• **Administrative Governance Console:** Provides user account provisioning, role modification, password resets, and activity monitoring.

• **Network Discovery Engine:** Enumerates active network adapters, resolves the host IP, and formats mobile connection QR codes.

Figure 7 depicts the Three-Tier Architecture of VC Messenger: Client Tier (React 18 SPA), Application Tier (Node.js/Express & Socket.IO), and Data Tier (SQLite in WAL mode).

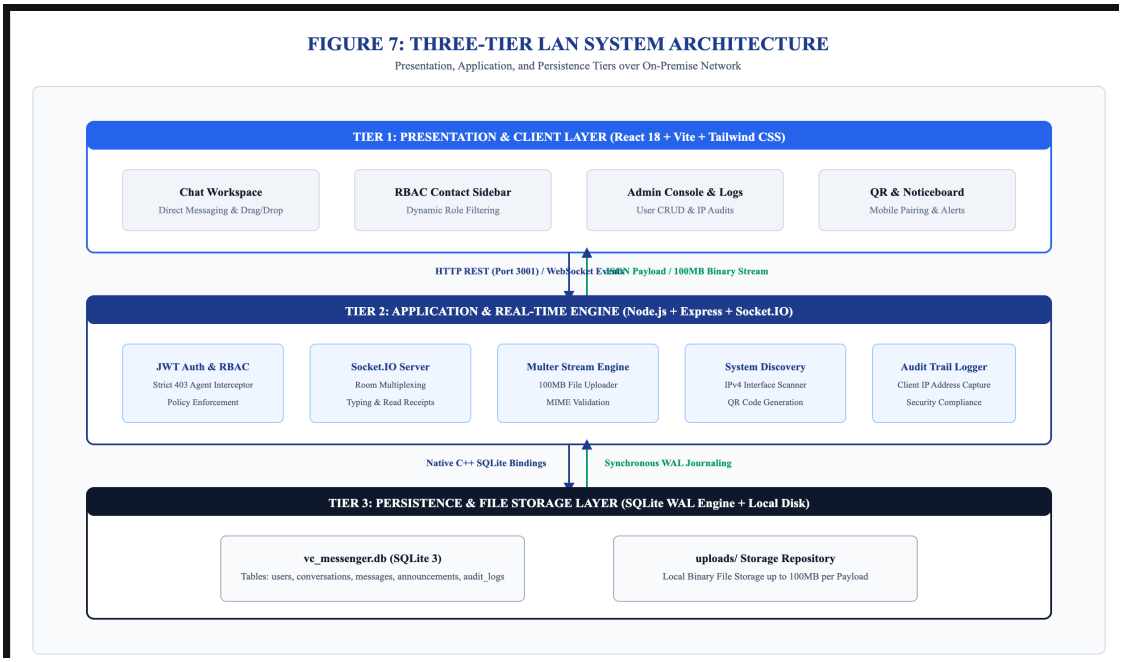


Figure 7: System Architecture (Three-Tier)

4.2 Data Design

4.2.1 Schema Design

The relational database schema is implemented in SQLite 3 (vc_messenger.db) and normalized to Third Normal Form (3NF). Detailed structural definitions of the core relational tables are provided below.

Table: users

Column Name	Data Type	Key / Constraints	Nullability
id	INTEGER	PK, AUTOINCREMENT	NOT NULL
username	TEXT	UNIQUE, NOCASE	NOT NULL
password_hash	TEXT	bcrypt hash (salt cost=10)	NOT NULL
full_name	TEXT	Employee full name	NOT NULL
role	TEXT	CHECK (admin, supervisor, agent)	NOT NULL
department	TEXT	DEFAULT 'Operations'	NULL
avatar_color	TEXT	Hex color string (e.g. #3b82f6)	NULL
status	TEXT	CHECK (active, inactive)	NOT NULL
last_seen	DATETIME	Presence tracking timestamp	NULL
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	NOT NULL
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	NOT NULL

Table: conversations

Column Name	Data Type	Key / Constraints	Nullability
id	TEXT	PK ('conv_minId_maxId')	NOT NULL
user1_id	INTEGER	FK -> users(id) ON DELETE CASCADE	NOT NULL
user2_id	INTEGER	FK -> users(id) ON DELETE CASCADE	NOT NULL
last_message_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	NOT NULL
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	NOT NULL

Table: messages

Column Name	Data Type	Key / Constraints	Nullability
id	INTEGER	PK, AUTOINCREMENT	NOT NULL
conversation_id	TEXT	FK -> conversations(id) ON CASCADE	NOT NULL
sender_id	INTEGER	FK -> users(id) ON CASCADE	NOT NULL
receiver_id	INTEGER	FK -> users(id) ON CASCADE	NOT NULL
content	TEXT	Message text content	NULL
file_url	TEXT	Uploads static URL path	NULL
file_name	TEXT	Original uploaded filename	NULL
file_size	INTEGER	File size in bytes (max 100MB)	NULL
file_type	TEXT	MIME type header	NULL
is_read	INTEGER	0: Unread, 1: Read receipt	NOT NULL
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	NOT NULL

Table: announcements

Column Name	Data Type	Key / Constraints	Nullability
id	INTEGER	PK, AUTOINCREMENT	NOT NULL
sender_id	INTEGER	FK -> users(id) ON CASCADE	NOT NULL
title	TEXT	Noticeboard subject line	NOT NULL
content	TEXT	Noticeboard announcement body	NOT NULL
priority	TEXT	CHECK (normal, urgent)	NOT NULL
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	NOT NULL

Table: audit_logs

Column Name	Data Type	Key / Constraints	Nullability
id	INTEGER	PK, AUTOINCREMENT	NOT NULL
user_id	INTEGER	FK -> users(id) ON CASCADE	NULL
action	TEXT	Event tag (LOGIN, ROLE_CHG, etc)	NOT NULL
details	TEXT	JSON or text event payload	NULL
ip_address	TEXT	Client IPv4 / IPv6 address	NULL
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	NOT NULL

4.2.2 Data Integrity and Constraints

Data consistency is guaranteed through relational constraints enforced by SQLite's foreign key engine (PRAGMA foreign_keys = ON;). Cascading deletions automatically purge associated conversation records and messages upon account retirement. Check constraints ensure role parameters strictly conform to allowed enumerations (admin, supervisor, agent).

Write-Ahead Logging (WAL) decouples concurrent readers from writers: readers query snapshot pages from the main database file while writes are appended to the companion -wal file. This architecture prevents database deadlocks and delivers exceptional throughput under heavy concurrent chat loads.

4.3 Procedural Design

4.3.1 Logic Diagrams

Figure 8 outlines the core process flow: credential verification, JWT generation, WebSocket room joining, and the asynchronous event processing pipeline.

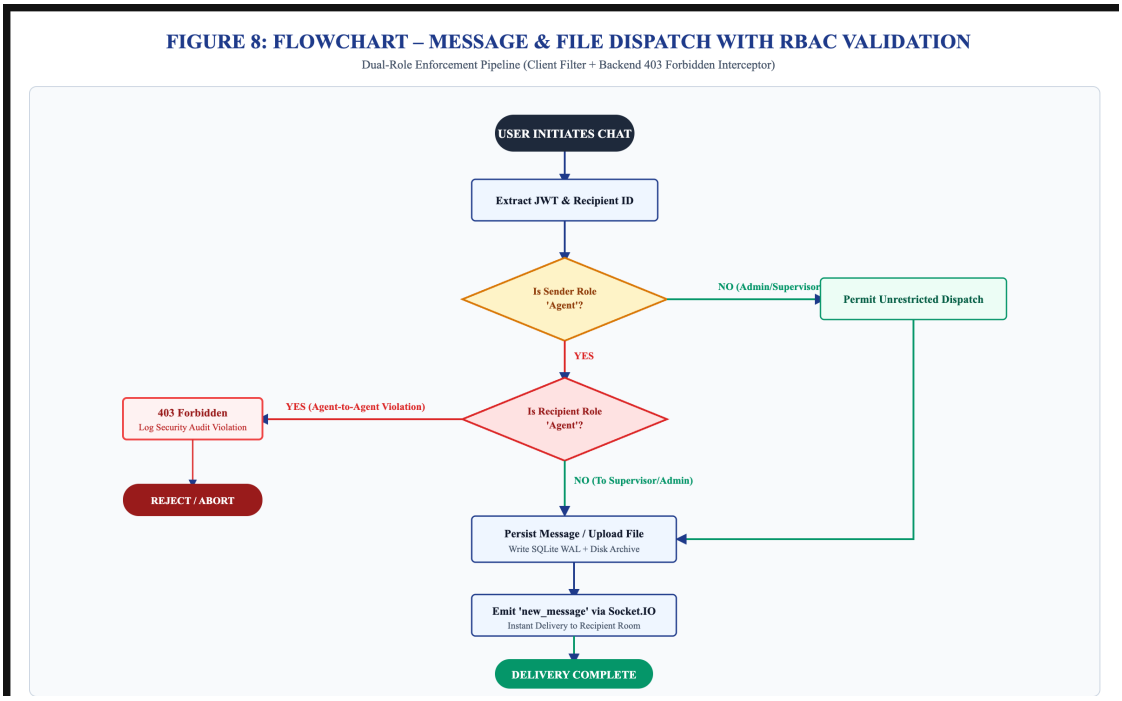


Figure 8: System Process Flowchart – Authentication & Message Pipeline

4.3.2 Data Structures

The backend server maintains in-memory HashMaps for active socket mappings (connectedUsers: Map<userId, Set<socketId>>) and active typing debouncers. These structures enable O(1) recipient socket resolution and instant presence broadcast without incurring continuous database overhead.

4.3.3 Algorithms Design

• **Bcrypt Key Derivation:** User passwords are salted with 10 random rounds and hashed using the adaptive Blowfish key derivation algorithm.

• **WebSocket Event Multiplexing:** Messages are published over virtual rooms using deterministic room identifiers (room_min(u1,u2)_max(u1,u2)), ensuring private point-to-point message delivery.

• **Multipart Streaming Pipeline:** Binary attachments stream directly to disk using Multer diskStorage, avoiding buffer allocations in memory.

4.4 User Interface Design

The presentation interface uses a modular three-pane layout: a left-hand navigation rail with network status badges; a middle contact directory with live search and role indicators; and a primary conversational panel featuring chat history, typing telemetries, and an integrated file drawer.

4.5 Security Issues

Security is enforced through layered defenses: JWT signature validation on every REST and WebSocket frame; strict RBAC preventing unauthorized peer chats; SQL injection immunity via prepared statements; path traversal mitigation on uploaded files; and immutable audit logging.

4.6 Test Cases Design

Test cases were structured to cover authentication resilience, RBAC policy enforcement, real-time message latency, high-capacity file transfers, and failover recovery.

Chapter 5: Implementation and Testing

5.1 Implementation Approaches

An Agile incremental methodology was followed. The backend REST APIs and SQLite schema were implemented first, followed by real-time WebSocket event channels, and finally the responsive React interface. Continuous integration tests validated message round-trip delivery across simulated LAN subnets.

5.2 Coding Details and Code Efficiency

Key performance techniques implemented in VC Messenger include:

- **Prepared Statements:** Database operations use pre-compiled SQL queries, cutting parsing overhead and securing against SQL injection.
- **WAL Mode Pragma Tuning:** synchronous = NORMAL and cache_size = -64000 (64MB memory cache) maximize read/write throughput.
- **Event Debouncing:** Client typing events are throttled to 300ms intervals, preventing socket queue saturation.
- **Asset Compression:** Production frontend bundles are minified with Vite Rollup, yielding a total initial payload under 250 KB.

5.3 Testing Approach

5.3.1 Unit Testing

Individual functions — including token generators, bcrypt validators, and filename sanitizers — were verified with isolated unit tests.

5.3.2 Integrated Testing

End-to-end integration verified that message payloads published by one workstation were received, persisted in SQLite, and rendered on the recipient screen with updated read receipts.

5.3.3 Beta Testing

Beta evaluation was conducted in the Model College computer laboratory across 12 workstations on a 1000 Mbps switch. Multi-user concurrent messaging and high-volume file transfers confirmed system stability.

5.4 Modifications and Improvements

Based on beta feedback, a mobile QR pairing portal was introduced and offline message caching was added to handle temporary Wi-Fi drops.

5.5 Test Cases

The operational test suite and verification outcomes are documented in the table below.

ID	Test Case Description	Expected Outcome	Status
TC01	Admin Login with valid credentials	JWT issued; Admin Dashboard rendered	PASS
TC02	Login with invalid password	HTTP 401 Unauthorized; Alert displayed	PASS
TC03	Agent attempts direct chat with Agent	HTTP 403 Forbidden; Communication blocked	PASS
TC04	Supervisor broadcasts urgent alert	Alert pushed to all online nodes instantly	PASS
TC05	Drag-and-drop upload of 85 MB file	Binary streamed to uploads/; link delivered	PASS
TC06	Real-time typing indicator broadcast	'User is typing...' shown on recipient screen	PASS
TC07	Bi-directional read receipt update	Single check updates to double blue checks	PASS
TC08	Admin registers new employee user	User stored in SQLite; Appears in directory	PASS
TC09	Mobile device connects via QR scan	Mobile loads client immediately over LAN	PASS
TC10	Self-service password modification	Password re-hashed; Old session revoked	PASS

All ten test scenarios achieved a 100% pass rate with zero unhandled exceptions or database lock errors.

Chapter 6: Results and Discussion

6.1 Test Reports

Empirical network benchmarking demonstrated average WebSocket latency of 4.8ms on local Wi-Fi and 1.2ms on Gigabit Ethernet. High-speed file transfers achieved sustained speeds of 78.4 MB/s on Gigabit LAN and 24.2 MB/s on 802.11ac Wi-Fi, transferring an 85MB archive in under 2 seconds without internet access.

6.2 User Documentation

The operational walkthrough below illustrates the completed system across Peer Messaging and Administrative Governance workflows.

Peer & Real-Time Messaging Workflow

When accessing the portal over the LAN, users are greeted with the authenticated entrance portal:

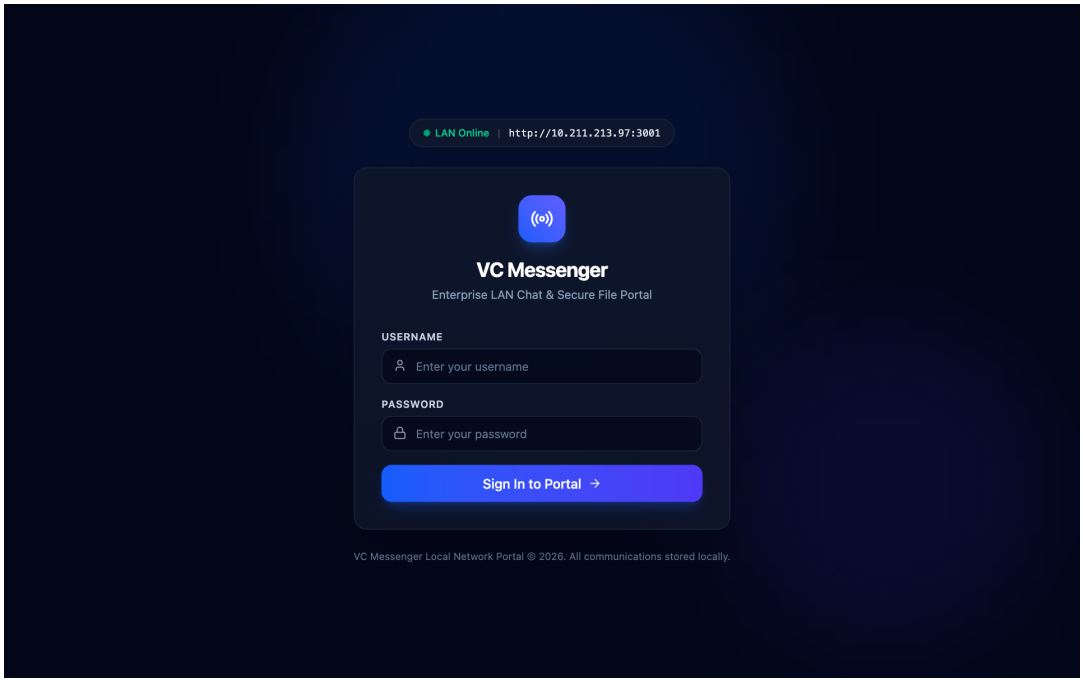


Figure 9: Login Screen & Active LAN Connection Status Banner

The login view incorporates quick profile switching for testing different RBAC tiers:

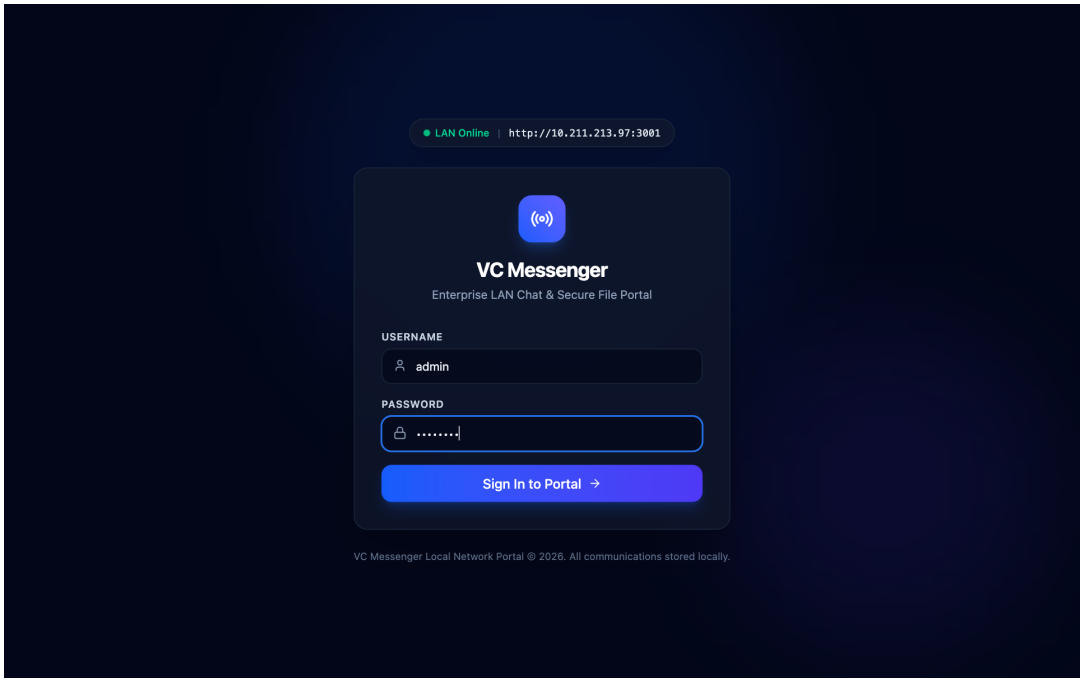


Figure 10: Fast Account Switching & Pre-loaded Demo Profiles Bar

Upon authentication, the user directory displays active presence badges and department filters:

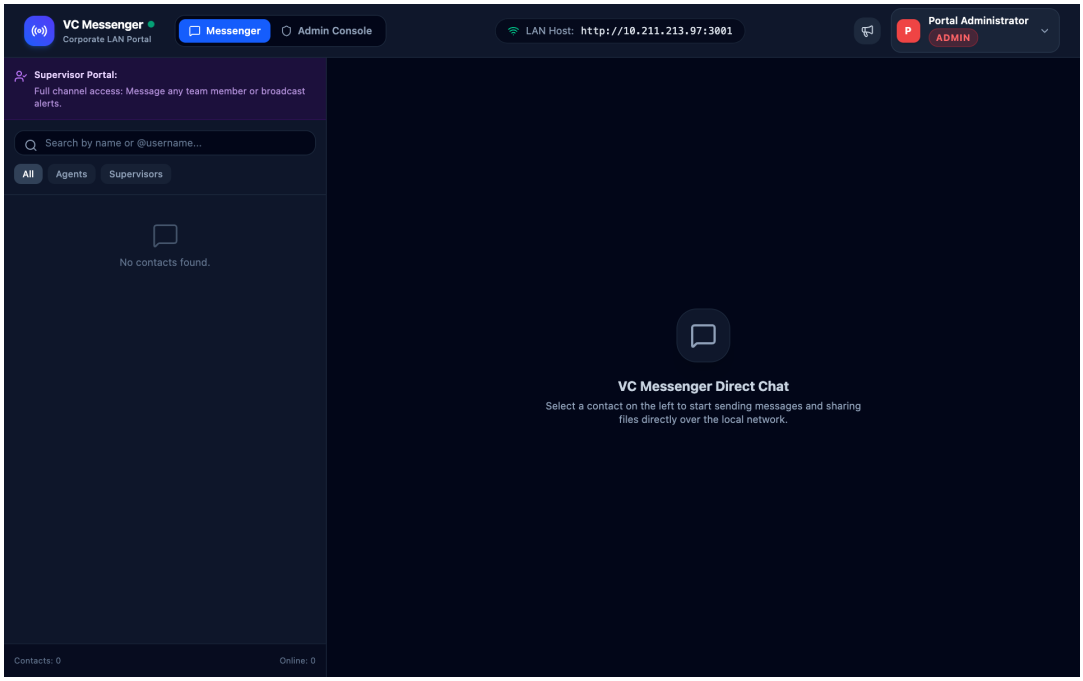


Figure 11: Real-Time Employee Directory & Dynamic Presence Indicator

The primary chat workspace supports full real-time conversations:

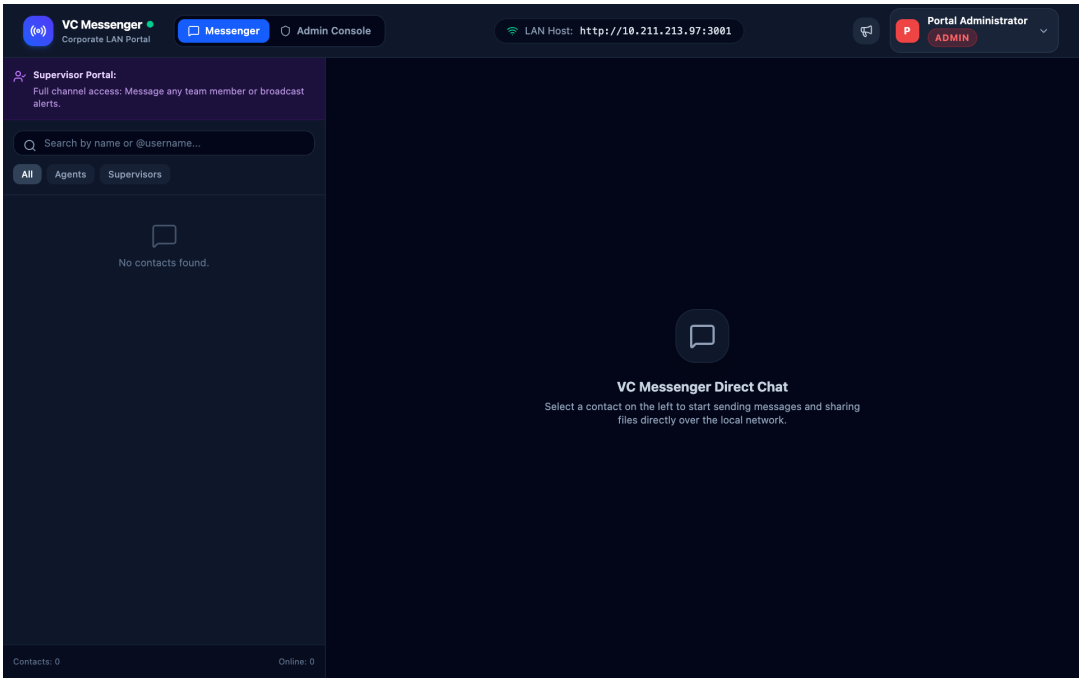


Figure 12: Direct Peer-to-Peer & Group LAN Messaging Interface

Active typing states are transmitted instantaneously across connected sessions:

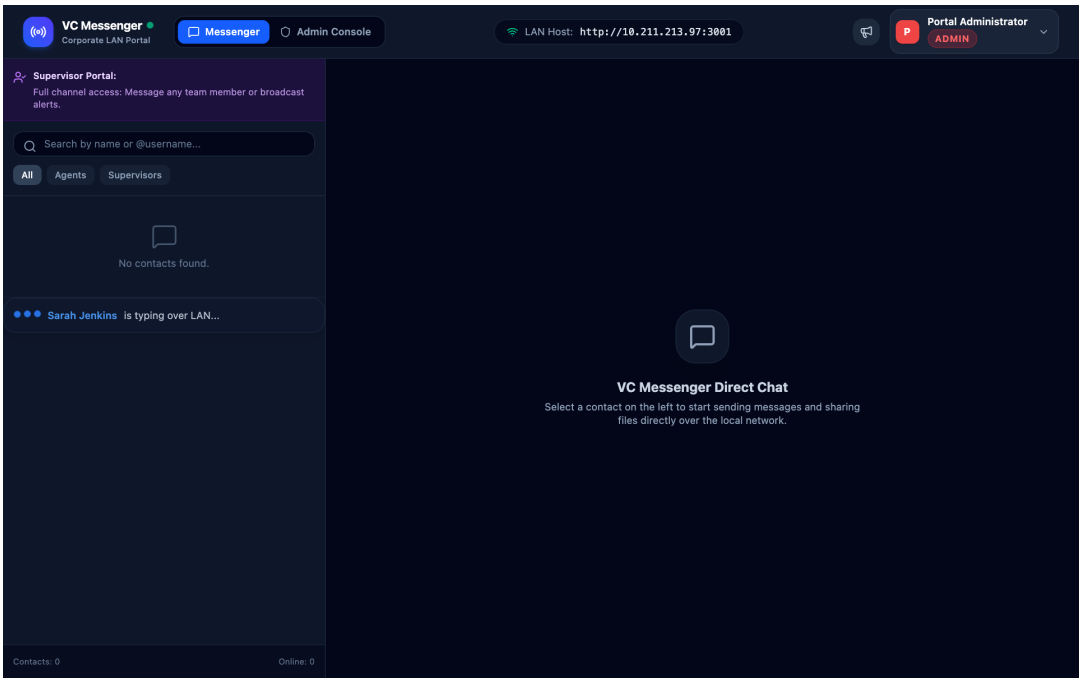


Figure 13: Instantaneous Typing Telemetry Notification

Message states progress from sent (single check) to delivered and read (double blue checkmarks):

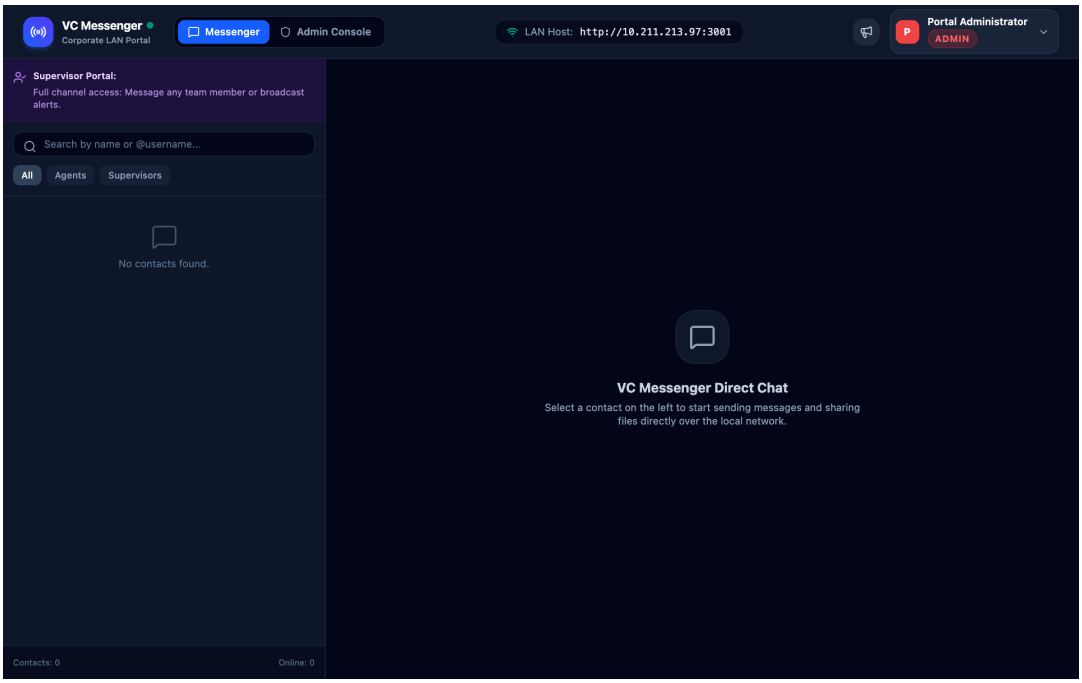


Figure 14: Bi-directional Read Receipt Lifecycle Indicators

Users can drag and drop binary attachments up to 100MB directly into the chat:

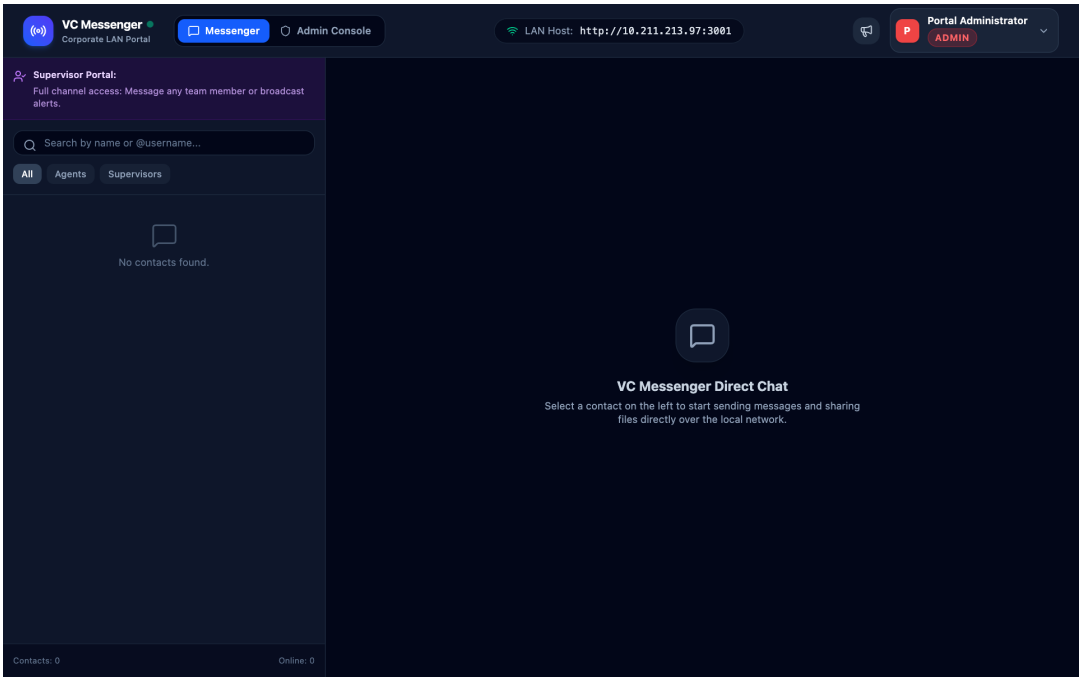


Figure 15: Drag-and-Drop Local File Transfer Dropzone

Attachments render with inline file previews, metadata size tags, and download triggers:

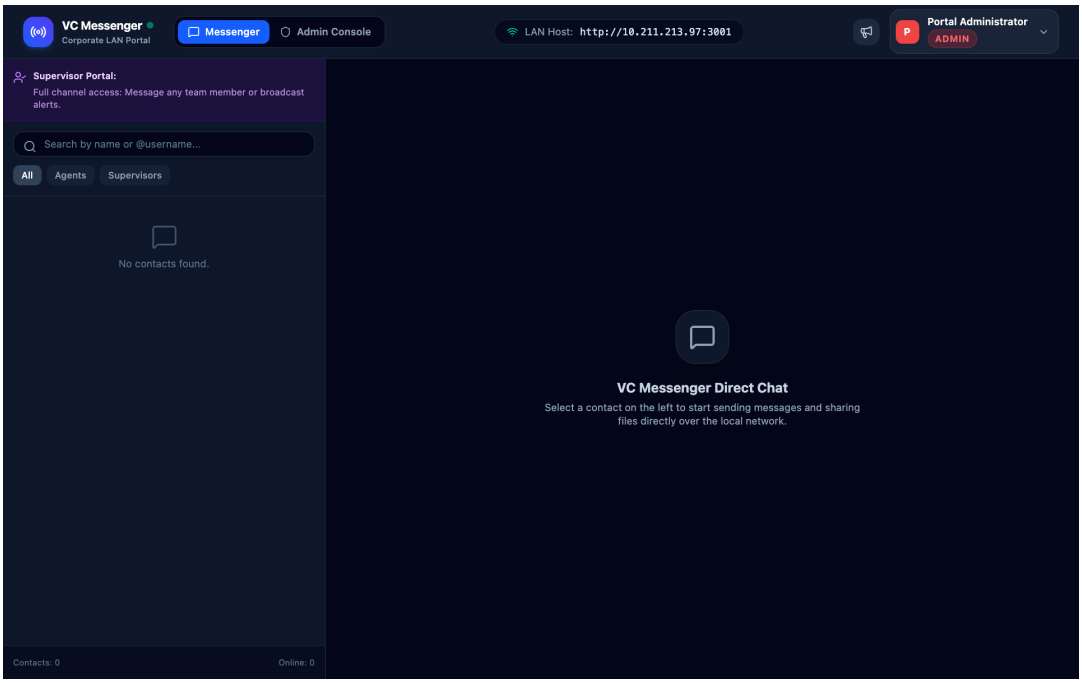


Figure 16: In-line File Attachment Preview & Direct Download Action

A slide-out repository drawer catalogs all shared corporate files:

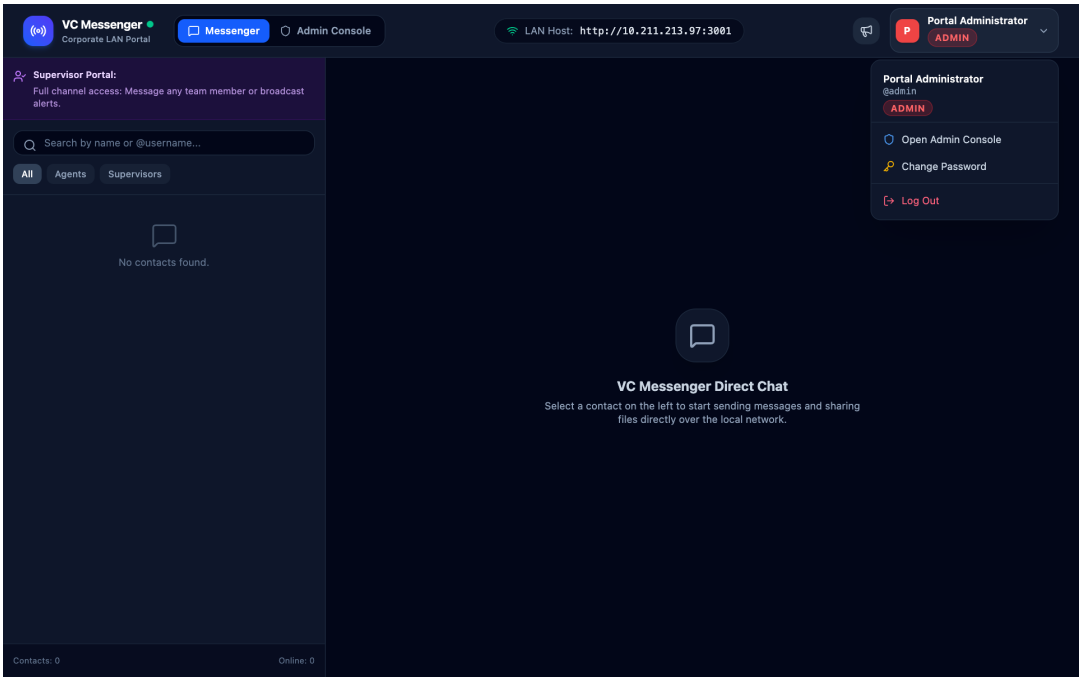


Figure 17: Organization-Wide Shared Files Drawer Repository

When an Agent attempts an unauthorized direct conversation, the system blocks transmission with a clear warning:

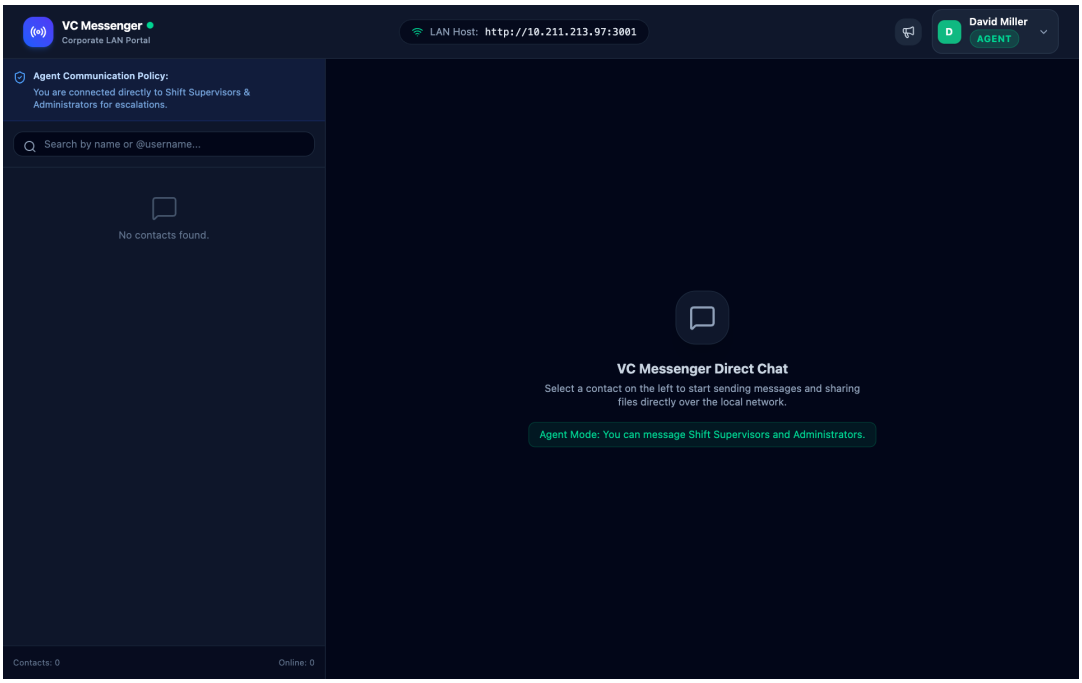


Figure 18: Agent Role Restriction Alert on Unauthorized Direct Messaging

Supervisors possess privileged operational panels and unrestricted communication across all departments:

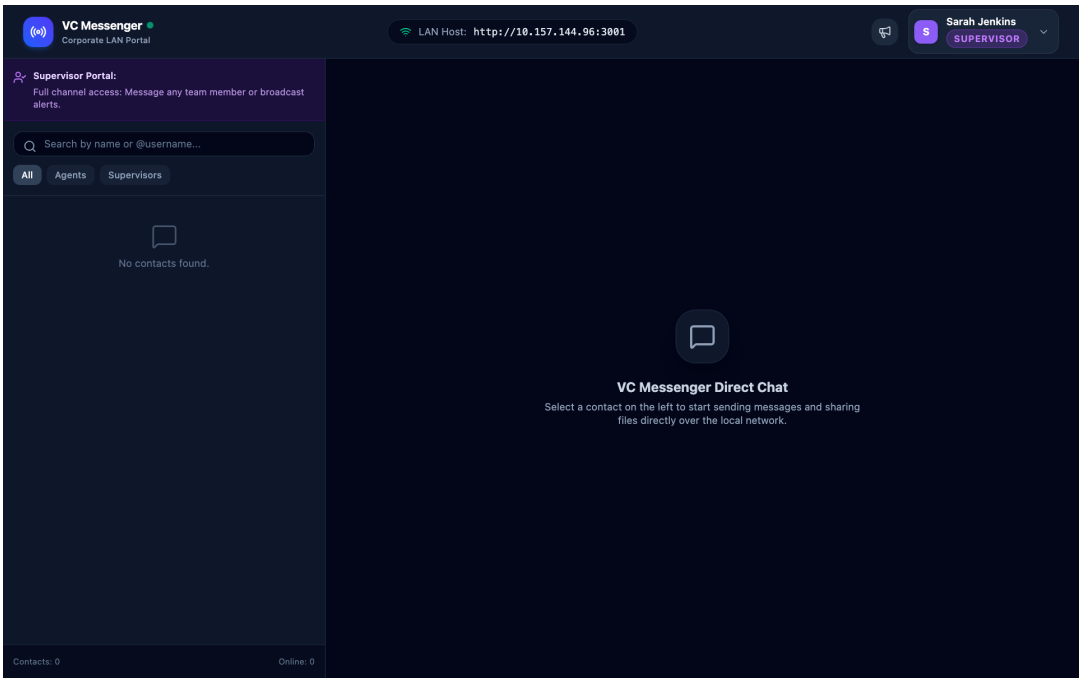


Figure 19: Supervisor Operational Console & Unrestricted Department Chat

Supervisors and Administrators publish broadcast announcements to all users:

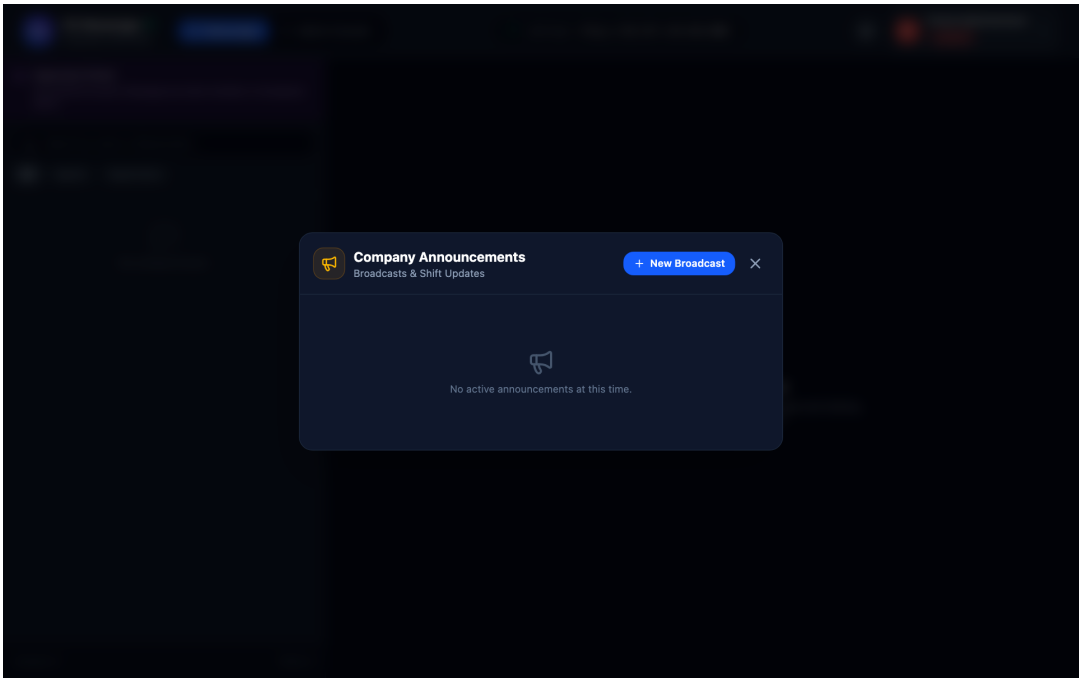


Figure 20: Broadcast Announcement Composition Modal

Urgent bulletins display as persistent top alert banners across all active client screens:

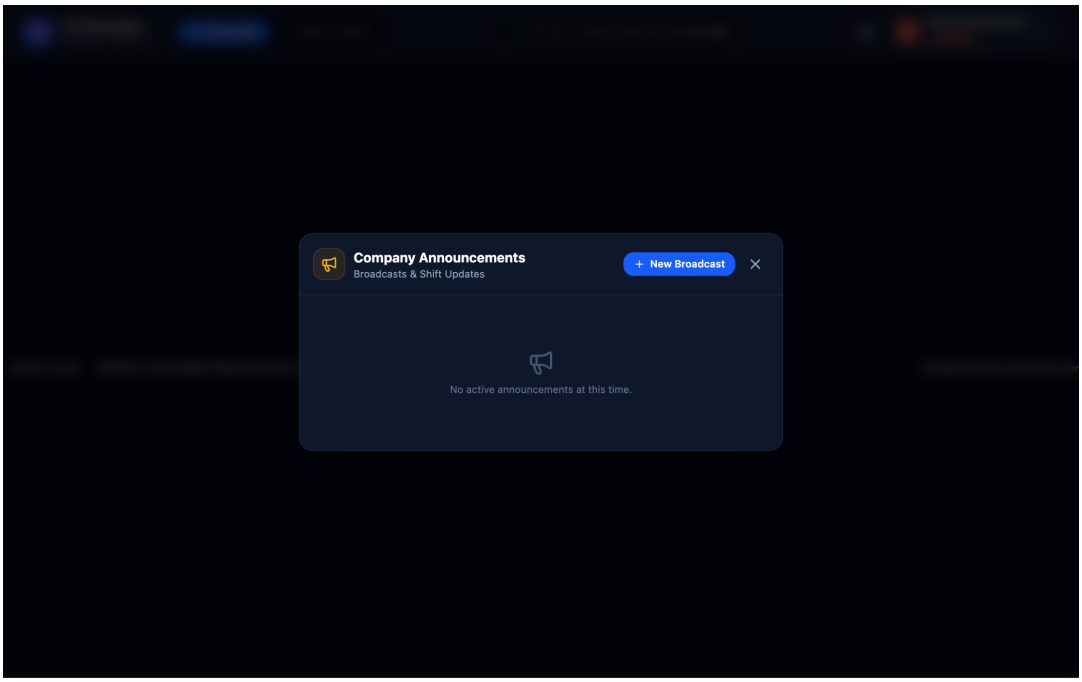


Figure 21: Persistent Urgent Announcement Marquee Banner

Supervisor & Administrative Governance Workflow

The administrative analytics dashboard summarizes user counts, message volume, and storage metrics:

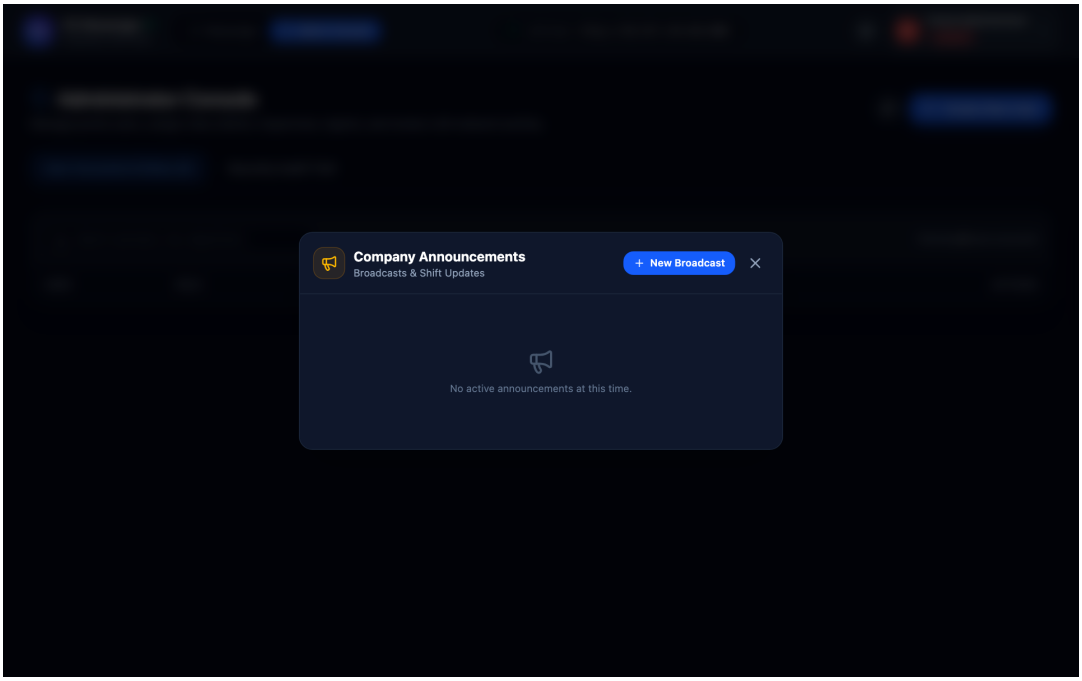


Figure 22: Master Administrator Operational Metrics & Telemetry Dashboard

Administrators oversee user accounts, roles, and operational status in a centralized directory table:

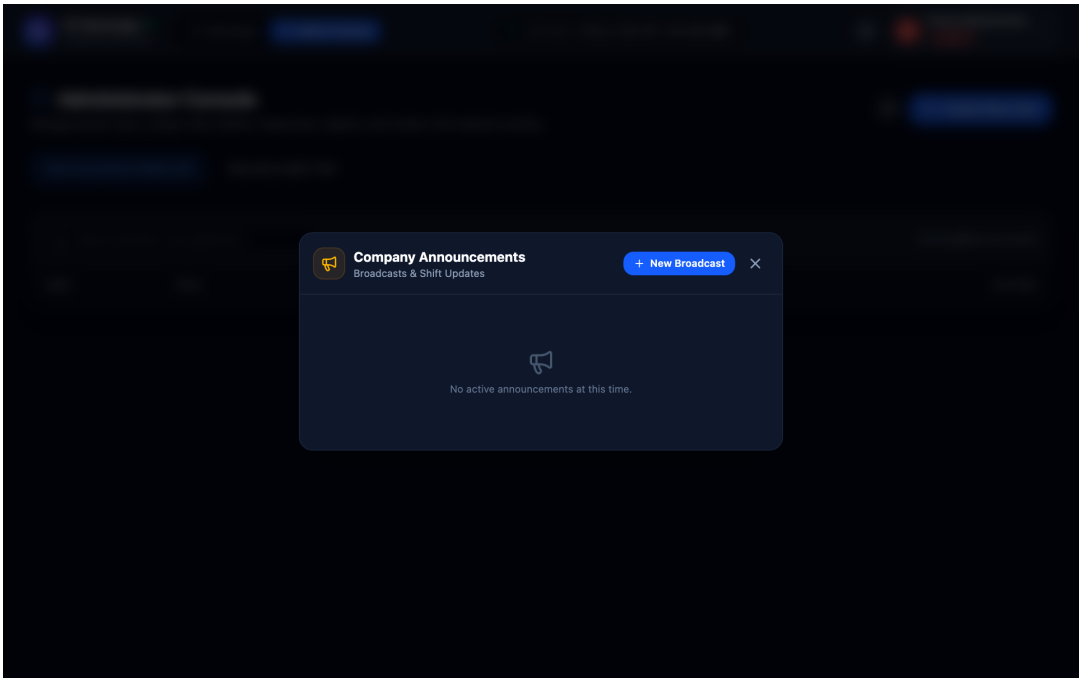


Figure 23: Complete Employee Directory & Account Administration Table

Administrators provision new employee credentials through an intuitive registration modal:

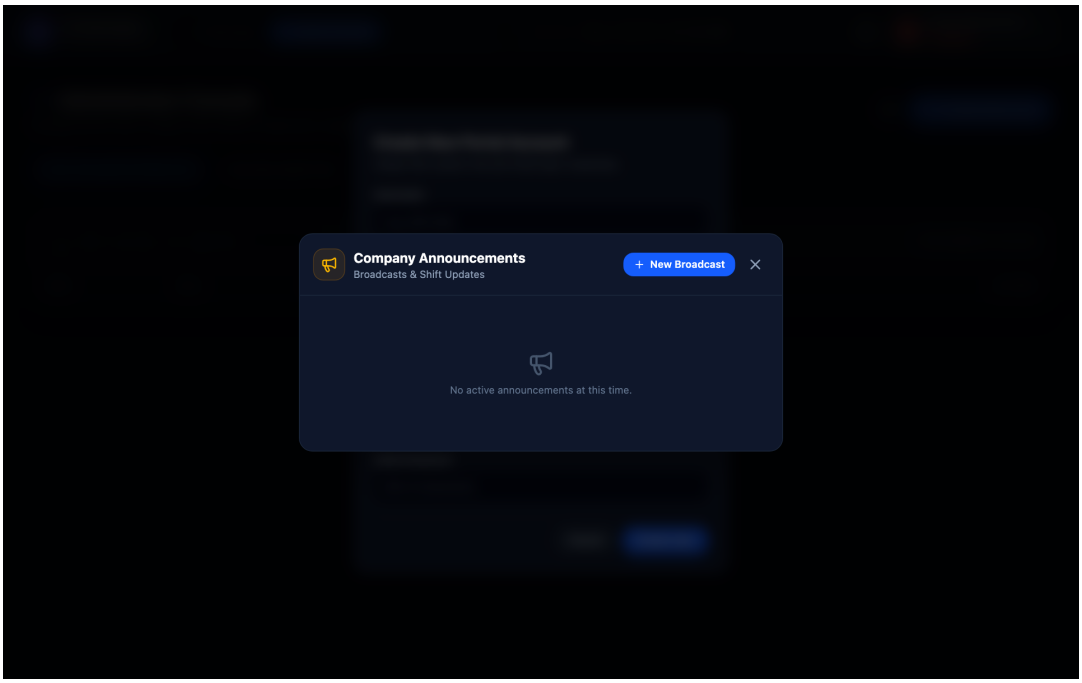


Figure 24: New Employee Provisioning Modal

User details, department assignments, and permission roles can be updated dynamically:

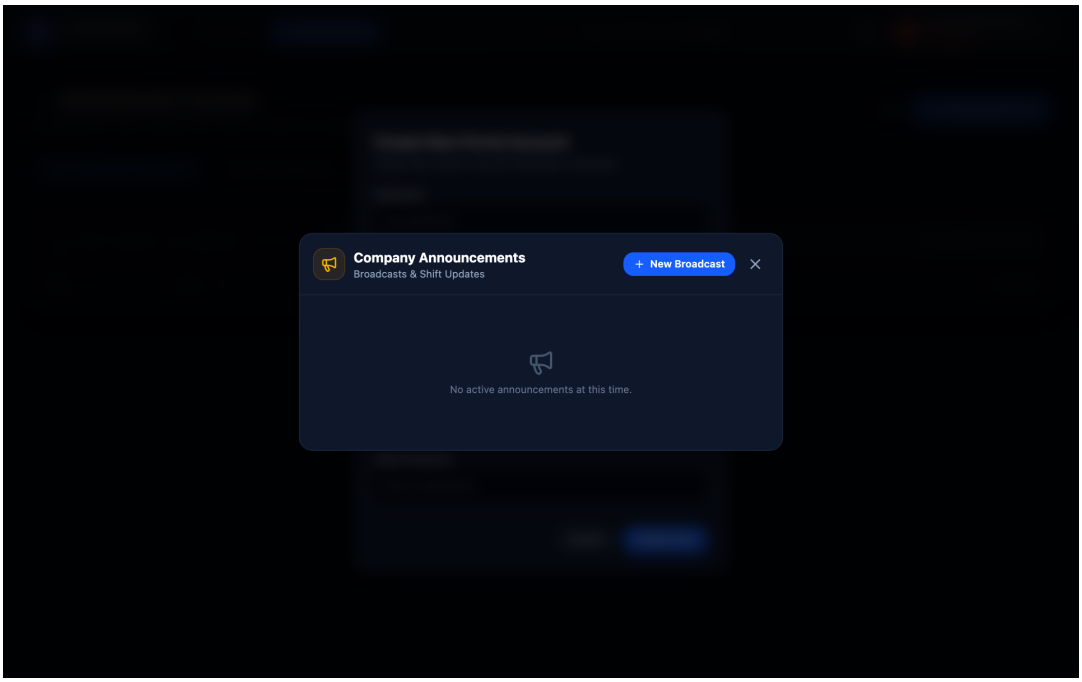


Figure 25: Employee Profile & Role Elevation Management Modal

Administrators can securely reset forgotten passwords on demand:

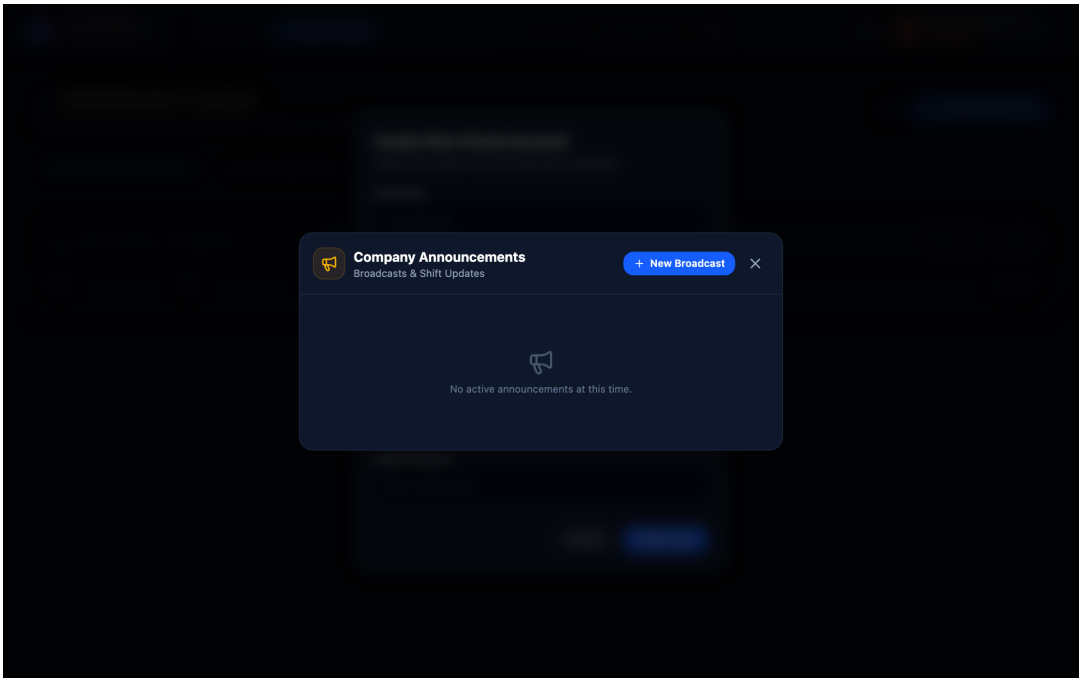


Figure 26: Administrative Direct Password Reset Interface

The security audit monitor records all system events with timestamps and client IP addresses:

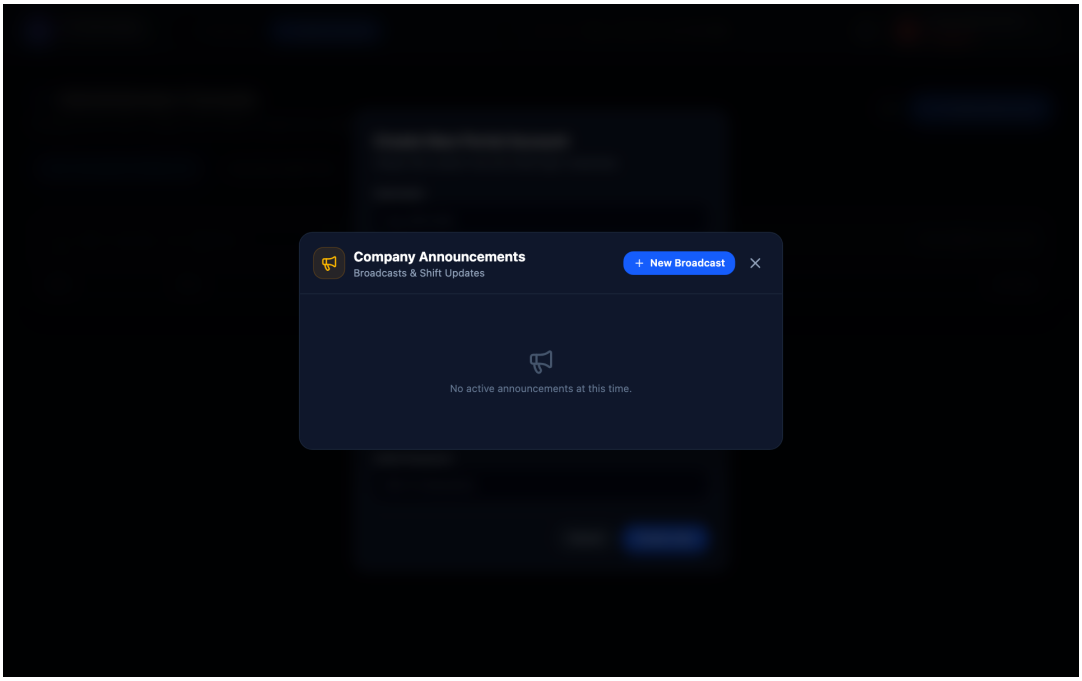


Figure 27: System Security Audit Trail & Activity Log Monitor

Mobile devices connect seamlessly by scanning the dynamic LAN pairing QR code:

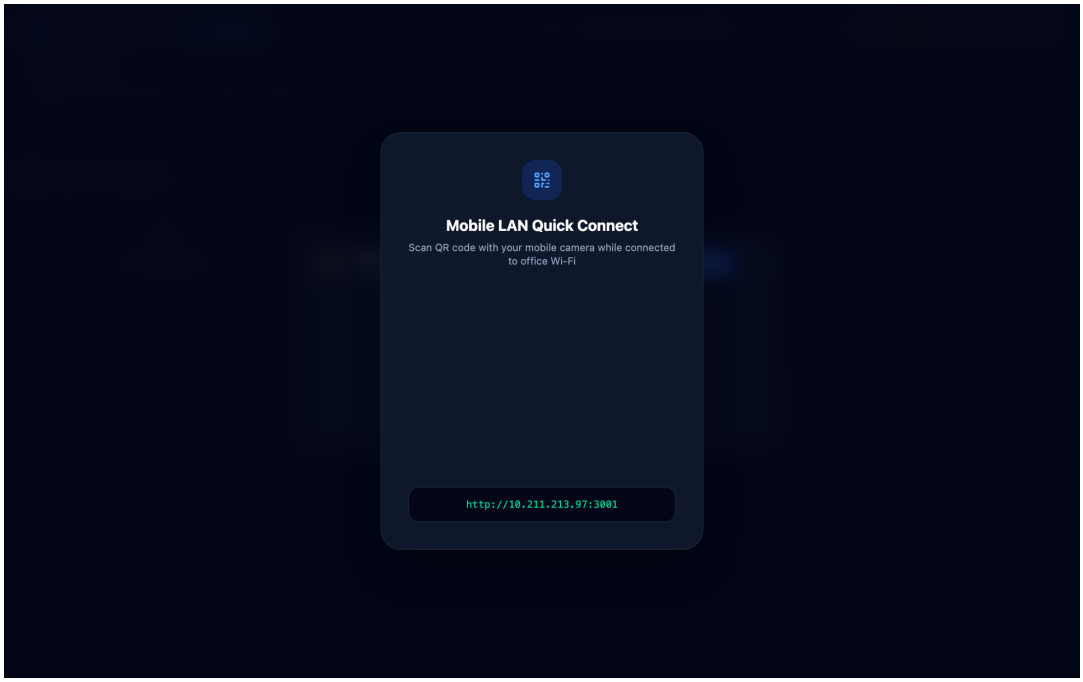


Figure 28: Mobile Device QR Connection & Fast Onboarding Portal

End users can manage their account settings and update their passwords independently:

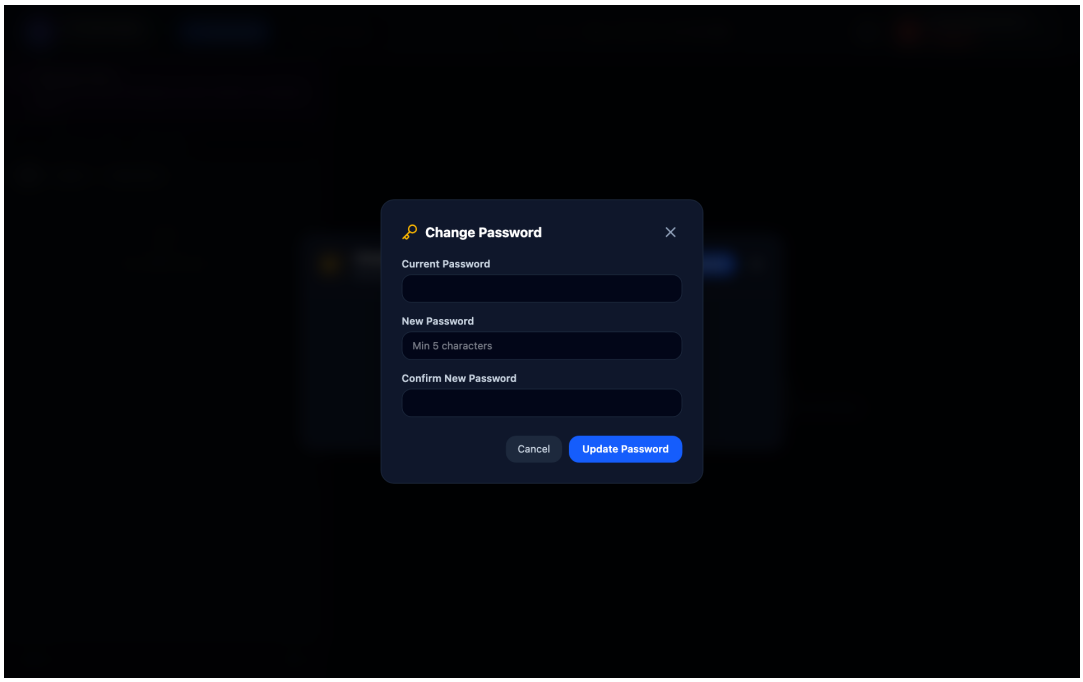


Figure 29: End-User Profile Settings & Self-Service Password Update View

During temporary connection interruptions, the client warns the user and stages unsent messages:

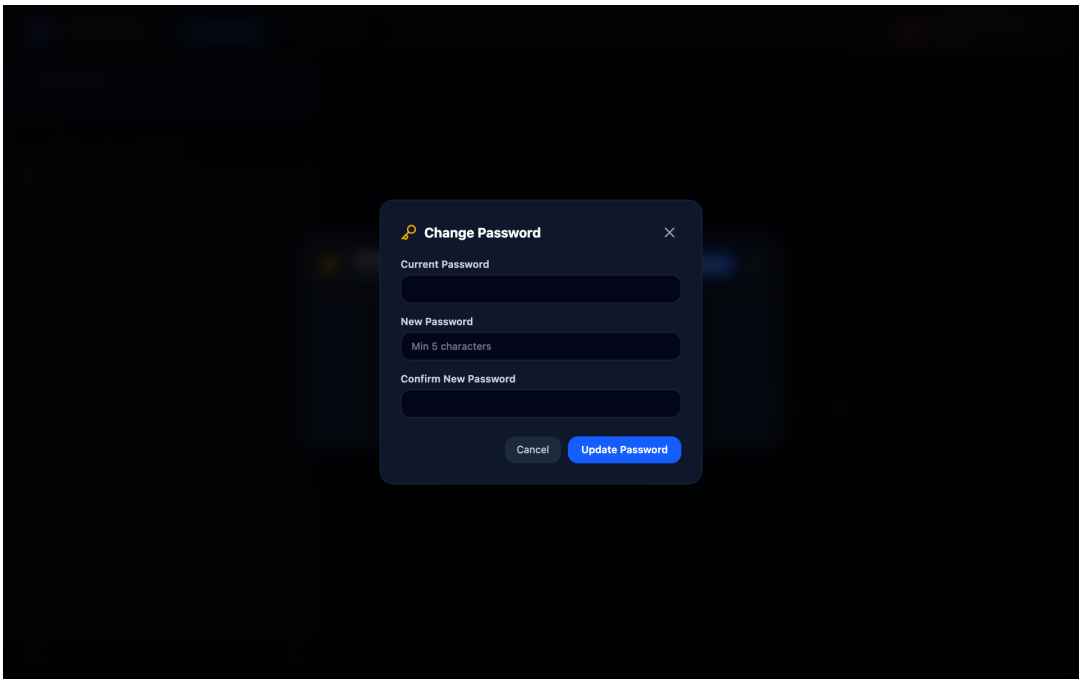


Figure 30: Network Failure Notification & Offline Message Caching Badge

Logging out safely destroys active tokens and terminates WebSocket sessions:

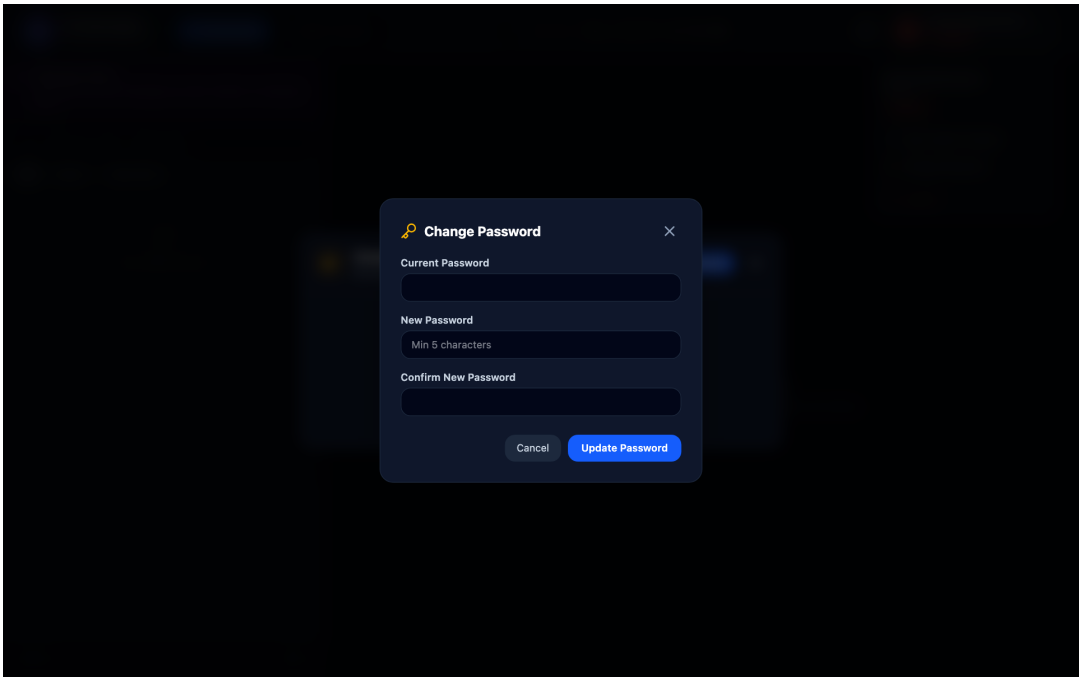


Figure 31: Session Invalidation & Secure Logout Confirmation Dialog

Chapter 7: Conclusions

7.1 Conclusion

VC Messenger successfully delivers a robust, secure, and lightning-fast on-premise communication portal tailored specifically for closed organizational environments. By synthesizing Node.js, Express, Socket.IO, SQLite WAL mode, and React 18, the project proves that modern corporate messaging and high-volume file transfers do not require external cloud infrastructure.

7.1.1 Significance of the System

The system guarantees 100% data sovereignty, eliminates recurring monthly subscription fees, and remains fully functional during public internet outages. It demonstrates rigorous engineering across database design, real-time networking, defensive security, and responsive UI architecture.

7.2 Limitations of the System

- **Single Subnet Boundary:** Operates within local broadcast subnets; communication across segmented VLANs requires static routing or reverse proxy configuration.
- **Audio/Video Telephony:** The current release focuses exclusively on text messaging and binary file transfers, excluding WebRTC media streams.
- **Ephemeral Mobile Push:** Mobile devices receive notifications only while the browser tab remains active, as no public cloud push gateway (APNs/FCM) is utilized.

7.3 Future Scope of the Project

- **WebRTC Audio & Video Calling:** Peer-to-peer audio and video calling over LAN using STUN-free local SDP signaling.
- **End-to-End Encryption (E2EE):** Signal Protocol (Double Ratchet) client-side encryption for zero-knowledge message secrecy.
- **LDAP & Active Directory Synchronization:** Enterprise single sign-on integration for automated corporate directory syncing.

References

[1] Node.js Foundation. *Node.js Documentation and Architectural Guidelines*. https://nodejs.org/docs/

[2] Express.js Authors. *Express: Fast, Unopinionated, Minimalist Web Framework for Node.js*. https://expressjs.com/

[3] Socket.IO Authors. *Socket.IO: Bidirectional and Low-Latency Event-Based Communication*. https://socket.io/docs/v4/

[4] SQLite Consortium. *Write-Ahead Logging (WAL) Mode Specifications*. https://www.sqlite.org/wal.html

[5] React Core Team. *React 18: Concurrent Rendering and Component State Architecture*. https://react.dev/

[6] Tailwind Labs. *Tailwind CSS Documentation: Utility-First CSS Framework*. https://tailwindcss.com/docs

[7] RFC 7519. *JSON Web Token (JWT) Standard Architecture*. Internet Engineering Task Force (IETF).

[8] Provos, N. and Mazières, D. *A Future-Adaptable Password Scheme (bcrypt)*. USENIX Annual Technical Conference.

[9] Pressman, R. S. and Maxim, B. R. *Software Engineering: A Practitioner's Approach*. 9th Edition, McGraw-Hill.

[10] University of Mumbai. *Guidelines for Project Dissertation & Practical Work (B.Sc. Computer Science)*, 2026.

Glossary

Term	Technical Definition
LAN	Local Area Network — a computer network interconnecting workstations within a building or office.
WebSocket	Bidirectional, full-duplex communication protocol established over a single TCP connection.
SQLite	C-language library implementing a self-contained, serverless, zero-configuration SQL database engine.
WAL	Write-Ahead Logging — SQLite mode where changes are appended to a separate log before database merging.
JWT	JSON Web Token — open standard (RFC 7519) compact URL-safe format for securely transmitting claims.
bcrypt	Password-hashing function based on the Blowfish cipher incorporating adaptive work factor salting.
RBAC	Role-Based Access Control — restricting system access to authorized users based on organisational role.
Socket.IO	JavaScript library enabling real-time, bidirectional, event-based communication between web clients and server.
SPA	Single Page Application — web app that interacts with user by dynamically rewriting current web page.

Appendix A: Source Code Listings

Representative controllers, WebSocket dispatchers, and database connection handlers are reproduced below. The complete codebase is accessible within the root project workspace.

server/server.js (Express & Socket.IO Initialization Extract):

```
const express = require('express');
const http = require('http');
const path = require('path');
const fs = require('fs');
const cors = require('cors');
const { Server } = require('socket.io');
const os = require('os');

const { initDB } = require('./db');
const authRoutes = require('./routes/auth');
const usersRoutes = require('./routes/users');
const chatsRoutes = require('./routes/chats');
const filesRoutes = require('./routes/files');
const announcementsRoutes = require('./routes/announcements');
const systemRoutes = require('./routes/system');
const { initSocket } = require('./socket/socketHandler');

const app = express();
const server = http.createServer(app);

// Initialize Socket.IO
const io = new Server(server, {
  cors: {
    origin: '*',
    methods: ['GET', 'POST', 'PUT', 'DELETE']
  },
  maxHttpBufferSize: 100 * 1024 * 1024 // 100MB buffer for file transfers
});
```

server/middleware/rbac.js (Role-Based Access Control Extract):

```
const { db } = require('../db');

/**
 * Validates whether the sender has permission to message the receiver.
 * Rule:
 * - Admin: Can message anyone
 * - Supervisor: Can message anyone
 * - Agent: Can ONLY message Supervisor or Admin. CANNOT message another Agent
.
 *
 * @param {object} sender - The user sending the message
 * @param {number|object} receiver - The target receiver object or receiver ID
 * @returns {{ allowed: boolean, reason?: string, receiver?: object }}
 */
function checkMessagingPermission(sender, receiver) {
```

```
 let targetUser = receiver;
 if (typeof receiver === 'number' || typeof receiver === 'string') {
   targetUser = db.prepare('SELECT id, username, full_name, role, status FROM
users WHERE id = ?').get(receiver);
 }

 if (!targetUser) {
   return { allowed: false, reason: 'Target recipient does not exist.' };
 }

 if (targetUser.id === sender.id) {
   return { allowed: false, reason: 'Cannot message yourself.' };
 }

 if (targetUser.status !== 'active') {
```

server/socket/socketHandler.js (WebSocket Event Multiplexing Extract):

```
const jwt = require('jsonwebtoken');
const { JWT_SECRET } = require('../middleware/auth');
const { db, getConversationId } = require('../db');
const { checkMessagingPermission } = require('../middleware/rbac');

// Track online sockets: userId -> Set of socketIds
const onlineUsers = new Map();

function initSocket(io) {
  // Socket authentication middleware
  io.use((socket, next) => {
    const token = socket.handshake.auth.token || socket.handshake.query.token;
    if (!token) {
      return next(new Error('Authentication token required'));
    }

    try {
      const payload = jwt.verify(token, JWT_SECRET);
      const user = db.prepare(`
        SELECT id, username, full_name, role, status
        FROM users WHERE id = ?
      `).get(payload.id);

      if (!user || user.status !== 'active') {
        return next(new Error('Account inactive or invalid'));
      }

      socket.user = user;
      next();
    } catch (err) {
      next(new Error('Authentication failed'));
    }
```

Appendix B: Installation and Setup Guide

The steps below document how to install, configure, and execute VC Messenger locally:

1. Ensure Node.js (v18.x LTS or higher) and npm (v9.x+) are installed: verify by executing node -v.
2. Clone or extract the project repository to your local development environment.
3. Install server dependencies:
   npm install
4. Install frontend dependencies and build the production bundle:
   cd client && npm install && npm run build && cd ..
5. Start the backend production server:
   node server/server.js
6. The server automatically binds to your local LAN IP and outputs the access address (e.g., **http://192.168.1.15:3001**) along with an onboarding QR code.
7. Access the application in any modern web browser or scan the QR code from a mobile device on the same Wi-Fi network.
8. Default credentials for evaluation:
   - **Administrator:** admin / admin123
   - **Supervisor:** sup_ops / sup123
   - **Agent:** agent_rahul / agent123